



DA7066: Programming techniques for Life Sciences

Lecture notes

Lars Arvestad, Christian Helanow, Anders Mörtberg, Kristoffer Sahlin

Contents

1	Getting started with Python	3
1.1	Programming and Python	3
1.2	Style	5
1.3	Variables	6
1.4	Working with strings and print	7
1.5	Arithmetic operations	8
1.6	Input	9
1.7	Special words and names in Python	9
1.8	Exercises	10
2	Python fundamentals	11
2.1	Syntax and semantics	11
2.2	Datatypes	12
2.3	Boolean expressions in Python	14
2.4	Assignment operators	16
2.5	Functions	16
2.6	Global and local variables	20
2.7	Conditional statements	22
2.8	Exercises	24
3	Lists, tuples, and iteration	26
3.1	Lists	26
3.2	Tuples	31
3.3	for loops	33
3.4	while loops	37
3.5	Exercises	40
4	Dictionaries, file handling, more on loops	42
4.1	Dictionaries	42
4.2	File handling	46
4.3	More on loops: nesting	48
4.4	Exercises	49
5	Programming in practice: algorithmic thinking, data representation, and debugging	52
5.1	Algorithmic Thinking	52
5.2	Data Representation	55
5.3	Debugging	58

5.4	Exercises	65
6	Error handling/exceptions	67
6.1	Error handling and exceptions	67
6.2	Exercises	75
7	Modules, libraries, and program structure	76
7.1	Modules	76
7.2	Popular libraries and modules	79
7.3	Rules of thumb for program structure and good code	85
7.4	Exercises	92
8	Writing a command line tool, testing	95
8.1	The terminal	95
8.2	Reading arguments from terminal	97
8.3	Argparse module	97
8.4	Testing	102
8.5	Exercises	104
9	Unix and the Terminal: Basic commands	106
9.1	Unix	106
9.2	Some commands	107
9.3	Running programs in background (&)	108
9.4	Signals:	108
9.5	Composing command lines	108
9.6	Streams	108
9.7	Pipes	109
9.8	Exercises	110
10	Data Analysis with Pandas, Numpy, and Scipy	111
10.1	Data analysis with Pandas, Scipy, and NumPy)	112
10.2	Why you still need classical coding skills	116
A	Sample solutions	119
A.1	Chapter 1	119
A.2	Chapter 2	120
A.3	Chapter 3	122
A.4	Chapter 4	126
A.5	Chapter 5	128
A.6	Chapter 6	129
A.7	Chapter 7	132
A.8	Chapter 8	136

Chapter 1

Getting started with Python

1.1 Programming and Python

These course notes are about *programming*, i.e., how to write code that a computer can execute to solve different computational problems. We will use the programming language *Python*, but many of the basic ideas are valid also for other programming languages (Java, JavaScript, C/C++, Rust, Ruby, Scala, Haskell, OCaml, ...). Python is very popular programming language (do a web search for “most popular programming languages”!) and has applications in just about any field. It is used in most companies, including IT giants like Instagram, Google, YouTube, Spotify, Amazon, et.c.

1.1.1 Installing Python and key packages

We will use version 3.x of Python. You can check if you already have Python installed on your machine by opening the *terminal* (in Unix or Linux) or *Command Prompt* on Windows. Then type `python --version` in your terminal/command prompt. It should return something like `Python 3.X.Y` (X and Y some numbers).

If you do not have Python on your machine, please install it. There are many ways to install Python (ask e.g., Google or ChatGPT). One option is to install *Anaconda* (<https://www.anaconda.com/download/success>), a tool for managing packages and environments for Python. The Anaconda distribution comes with several Python packages that are not included in the standard Python distribution, such as `matplotlib`, `numpy`, and `pandas`. We will use some of these packages later in the course.

In case you opt to not install the Anaconda distribution, you can install these packages with Python’s built-in package installer (`pip`). In your terminal, type

```
python -m pip install --upgrade pip
python -m pip install numpy pandas matplotlib
```

You can verify the installation by typing the following in your terminal/command prompt):

```
python
import numpy
import pandas
import matplotlib
print("NumPy version:", numpy.__version__)
```

```
print("Pandas version:", pandas.__version__)
print("Matplotlib version:", matplotlib.__version__)
```

1.1.2 Deciding your coding environment

So, where do we write the Python code? Essentially, we can write code in any text editor (e.g., Microsoft Word or a TextEdit if you will) as long as the file with Python code we save ends with `.py` (i.e. a *Python file*). However, coding in such a text editor is not very productive, as they are designed for writing plain text and do not have features that support coding. Instead, there are plenty of good *programming editors* or *coding editors* that support *syntax highlighting* and other tools that make programming easier. An *Integrated Development Environment (IDE)*, is an even more extensive *programming editor* that supports large programming projects with external tools to check performance (profiling) and find errors (debuggers), but they can sometimes be harder to use.

If you already have experience with coding in Python and feel comfortable with a specific programming editor, feel free to use it in this course. If not, we will use the programming editor *Visual Studio Code (VS Code)* that can be used both as a simple programming editor and an IDE. Download and install VS Code through <https://code.visualstudio.com/download>. VS Code has become the most widely used Python editor and combines advanced features, flexibility, and ease of use. We won't be using any advanced features in this beginners course, but instead use it primarily as follow-up courses use it.

If you are curious about other coding editors, here are some options:

- Spyder - a favourite among beginners. Easy and intuitive, uniquely designed for scientific computing and exploration (a MATLAB-like experience)
- PyCharm Community Edition — “community edition” is free, good integration with Python.
- Atom.io
- Sublime
- BBEdit — limited version is free
- For enthusiasts, there are two classics with a higher learning threshold:
 - Emacs
 - Vim

1.1.3 Running Your First Python Program in VS Code

In case you decide not to use VS code in this course, you can skip this section. When programming in Python, you write code in a file with the suffix `.py`. Your code is then sent to a Python *interpreter* that reads and executes the code.

1.1.3.1 Open VS Code

1. Open *Visual Studio Code* from your Start Menu (Windows) or Applications folder (macOS). You should see the VS Code welcome screen.
2. Install the Python Extension
 - Click the Extensions icon on the left sidebar (or press Ctrl+Shift+X on Windows or Cmd+Shift+X on macOS).
 - In the search bar, type Python.
 - Install the extension published by Microsoft.
 - Restart VS Code if prompted.

This extension provides Python syntax highlighting, code completion, and the ability to run Python scripts directly inside VS Code.

3. Create a New Python File

- Go to File -> New File.
- Save it as `hello.py` (choose a folder you can easily find, we recommend creating a DA7066 folder for all materials in this course). The `.py` extension tells VS Code that this is a Python file
- Type the following code `print("Hello World")` in the file.

4. Select Python Interpreter: VS Code needs to know which Python to use:

- Press `Ctrl+Shift+P` (Windows) or `Cmd+Shift+P` (macOS) to open the *Command Palette*.
- Type Python: Select Interpreter -> press Enter.
- Choose the version of Python you installed (e.g., Python 3.11 from Anaconda/Miniconda or system Python).

This ensures VS Code runs your code with the correct Python version.

5. Run the Program. There are two ways:

- Option A: Run Using the “Run” Button. You should see a small triangular run button in the top right of the editor. Click it — the output (Hello World) will appear in the terminal window below.
- Option B: Open the terminal window yourself by opening the Command Palette (`Cmd+Shift+P` for Unix/Mac and `Ctrl+Shift+P` on Windows) and typing “Terminal” and press enter. Then, in the terminal type `python hello.py`.

The output in both cases will be

```
SU-MacBook-M1:~ ksahlin$ /usr/local/bin/python3 /Users/ksahlin/Downloads/hello.py
hello, world!
```

The first line specifies the path to the Python Interpreter and the path to the file we created. The second line is the output of the code.

Note: In this text, output from computations are written with a turquoise background.

1.2 Style

1.2.1 Representing code

This is a course in English, so of course that will be the main language. However, for coding in general, it is advised to always use English. This is common practice, in part because coding is often done in international environments. Many programmers prefer English also in very national environments since the programming language is in English.

When it is obvious that we are writing code in the editor and executed in the terminal, we use gray and turquoise backgrounds, like this:

```
print("Welcome to DA7066")
print(2*2)
```

```
Welcome to DA7066
4
```

1.2.2 Assignments

For problems in this text, the symbol † mark exercises that we believe are harder.

1.2.3 Comments

It is important to document ones code with informative comments. In code, comments are started with the sign #. It is good practice to use comments to give context to what the code is supposed to do, and add information that is otherwise hard to deduce from the actual code. The main goal with comments is, of course, to make it easier for readers of the code to understand it. It is our own experience that it can be very hard to come back to ones own code, so this potential future reader of the code might be yourself! Many, many, programmers have looked at their code some time later and asked “what was I thinking? What is happening here?!” So, write comments to help your future self!

An example:

```
print('This is printed') # This comment is not printed
```

```
This is printed
```

The following are examples of how to comment code:

```
print(17) # Prints the number 17
print(1.1) # Prints the number 1.1
print('DA7066') # Prints the course code for this course
```

One can note that these examples are not very good, because they are explaining what also a beginner might feel is the obvious outcome of the code. Good comments do not state the obvious, but explains *why* the code is written as it is.

1.3 Variables

Variables are used to store values of different sorts.

```
x = 17
y = 'DA7066'
z = 22/7
q = 22//7

print(x)
print(y)
print(z)
print(q)
print(x,y,z,q)
```

```
17
'DA7066'
3.14
3
17 DA7066 3.142857142857143 3
```

Note that we have to use print for the values to show up in the console.

The example above is using these types of values:

- integer, abbreviated to `int` in code
- floating point, abbreviated to `float` in code
- strings (for text)

These are called “primitive types” in Python, and we will come back to what that really means.

You can also assign several values at once:

```
x,y = 2,3
print(x)
print(y)
```

```
2
3
```

Note that if `x` and `y` has already been defined in the code (e.g., as 17 and 'DA7066' earlier), then they will be overwritten with the new values 2 and 3.

1.4 Working with strings and print

Strings can be written using single quotes `'` or double quotes `"`. This redundancy makes it easier to add single or double quotes in strings:

```
print("hej")
print('hej')
print("h'e'j")
print('h"e"j')
```

```
hej
hej
h'e'j
h"e"j
```

You can also use backslash, `\`, to “protect” the quotes:

```
print('h\'e\'j')
print("h\"e\"j")
```

```
h'e'j
h"e"j
```

Line breaks are written using `\n`:

```
print("h\n e\n j")
```

```
h
e
j
```

If you want to print several values with blank spaces in between, you simply call `print` with multiple parameters:


```
print('hello', 'how', 'are', 'you')
```

```
hello how are you
```

1.5 Arithmetic operations

Python supports the usual arithmetic operations:

- addition (+)
- subtraction (-)
- multiplication (*)
- division (/)
- integer division (//)
- exponentiation (**)
- modulo/remainder (%)

Try it!

```
2+3 # yields 5
2-3 # yields -1
2*3 # yields 6
2/3 # yields 0.6666666666666666
2//3 # yields 0
2**3 # yields 8
2%3 # yields 2
```

This also works for *floating point numbers*, the closest thing you get to the real numbers in a computer.

There are also operations on strings. For example, + concatenates strings:

```
'Mid' + 'summer' # results in 'Midsummer'
```

If you want a blank space in between words, you have to add it yourself:

```
'hi' + ' ' + 'there' # results in 'hi there'
```

The property that + can be used in different ways with different types of values makes it an *overloaded operator*.

The arithmetic operations work mostly as expected (e.g., associative rules are in effect, and parenthesis can be used), so $2 * 3 + 4$ is the same as $(2 * 3) + 4$:

```
2 * 3 + 4 # yields 10
(2 * 3) + 4 # yields 10
2 * (3 + 4) # yields 14
```

For numerical values, mixing types have (mostly) the expected effect:

```
2.0 ** 3 - 2.1312 # yields 5.8688
```

But working with strings gives operators novel meanings:

```
2 * 'two' # yields 'twotwo'
```

And you cannot add a number to a string, nor multiply strings:

```
'hi' * 'there'
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: can't multiply sequence by non-int of type 'str'
```

```
2 + 'hej'
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Note the informative error messages! A productive programmer studies and ponders error messages. They are sometimes pinpointing the problem for you, other times at least giving hints as to what the real problem is.

1.6 Input

We have seen that `print` can be used to output values. For reading from the keyboard, you use `input`. Using the statement `input()` alone simply waits for the user to write something and hit return, but if you give a string as parameter, then that is printed first:

```
name = input("What is your name?")  
print("Hi " + name + "!")
```

Running this code in the console looks like:

```
What is your name?  
Xena # typed by you!  
Hi Xena!
```

1.7 Special words and names in Python

1.7.1 Reserved words

There are a number of reserved words, or *identifiers*, in Python:

```
and except lambda with as finally nonlocal while assert false None  
yield break for not class from or continue global pass def if raise  
del import return elif in True else is try
```

These identifiers are part of the language. Using a reserved word as a variable identifier results in an error:

```
and = 12
```

```
File "/Users/ksahlin/DA7066/hello.py", line 1  
  and =12  
  ^  
SyntaxError: invalid syntax
```

There are quite few restrictions on how to choose identifiers.

1.7.2 Identifiers

Identifiers are names for resources, such as variables. Resources can also be in the form of functions (chapter 2), classes, objects, etc. Classes and objects are not part of this course.

Python has these rules for identifiers:

- May contain letters, digits and underscore (_).
- Cannot begin with a digit.
- Python is case sensitive; Year and year are not the same identifier.

Make sure to choose good identifiers to make the code easier to read! Naming a variable for someone's age as age is much more informative than x.

1.7.3 Literals

A literal is a constant value in code. For example, 17, 2.5, and 'SU' are literals.

1.8 Exercises

1. What happens if we run the code below? Why?

```
x,y = 2,3
x,y = y,x
print(x)
print(y)
```

2. Below is code to define the variables x and y with assigned values. Create and print a variable z with the value 100 by *combining* x and y arithmetically.

```
x,y = 4, -3
```

3. Let

```
s = 'hej'
```

Write code to print 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' (i.e., sixteen copies of hej).

4. Let

```
s = 'hej'
```

Write code that results in 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' by using the + operator as few times as possible.

5. Write code that asks the user for first and last name, stores them in variables, and then prints a greeting. Look at the section section 1.6 above for inspiration.

Example session: python What is your first name? What is your last name? Hi Xena Warriorprincess!

6. Write code that asks the user for his/her age and favorite number and store them in two variables. What happens if you add those values and prints the result? What did you expect and what can you conclude about age and num?

Chapter 2

Python fundamentals

2.1 Syntax and semantics

In programming, we often talk about *syntax* and *semantics*:

- *Syntax* deals with rules for what one *writes*
- *Semantics* deals with the *meaning* of what one writes

As an example, consider the following sentence:

Colorless green ideas sleep furiously.

The sentence is syntactically correct: there is no *grammatical* error. But the sentence is not *meaningful*: the semantics is nonsense.

Syntax is especially important, and can be difficult to grasp, when one begins to learn to program. Just like with an ordinary language, one has to learn the rules for what is correct. In the case of Python, you need to learn for example what is a valid name for an identifier, where and how you should indent a block of code, how to write *if*-statements and *for*-loops (we will see these in later chapters), and so on. Every programming language has its own syntax, though there are often similarities. (Python, for example, has a lot in common with the language C.) You can get used to a language's syntax relatively quickly with practice, though, and then the focus shifts to semantics: what does a program do, and what do you want it to do?

If you see the label `SyntaxError` in an error message when you execute Python code, it means you have written something incorrectly and so Python cannot understand your code. In this case, you often get a fairly informative error message, as Python can point out the place where it got confused. For example, you might see the following message:

```
3a = 2
  ^
SyntaxError: invalid syntax
```

The error message shows where Python found invalid syntax, here an invalid name (3a) for an identifier.

Semantic errors are harder to find, because Python itself does not know if something is wrong. The code is all valid: the result of a semantic error is that the program doesn't do what you expected. You can reduce the chance of a semantic error by writing well-organized, well-documented, and well-structured

code, that is to say “good” code. If it is clear what you *want* the code to do when someone (or you yourself) reads the code, it is much easier to find semantic errors.

2.2 Datatypes

Every value in Python has a type. We have already used different types when we for example assigned variable names to integers, floats, and strings. The most common types in Python we will use are:

- `str` – strings, that is letters (or symbols more generally)
 - Written in between ' or "
- `int` – integers, of unlimited size
 - Written using digits 0-9
 - In many other languages, `int` is the type of 32-bit integers, specifically integers in the range $[-2^{32}, 2^{31}-1]$
 - (before Python v3.0, `int` was the type of integers up to 32-bits, while unlimited-size integers were called `long`)
- `float` – floats, an approximation of a real number
 - Written using digits 0-9 and a decimal point .
 - Typically numbers in the range $[-10^{308}, 10^{308}]$, with 15-digit precision
 - 64 bits, 11 for the exponent e and 52 for the “mantissa” m
 - Details can be found by running `import sys` and looking at the object `sys.float_info`
 - Why isn’t the result always exactly what you expect? Answer: Python represents floats using base 2 fractions. For details, see: <https://docs.python.org/3/tutorial/floatingpoint.html>
In general, it is not possible to represent *all* real numbers *exactly* on a computer, no matter what representation one uses. *Some* real numbers can be represented exactly while others must be approximated: more bits results in a better approximation.
- `complex` – written `1+2j`
 - Represented by two floats
- `bool` – Boolean values (that is, ‘true’ or ‘false’), represented by the reserved keywords `True` and `False`.
- `NoneType` – for the value `None`

In Python, you can see which type a variable or literal has by using `type`:

```
my_int = 1
my_float = 1.0
my_string = "1"
type(my_int)
type(my_float)
type(my_string)
type('hello')
```

```
int
float
str
str
```

2.2.1 Type coercion

You can convert a value of one type to another, but for each pair of types, there are different rules for whether one can convert between them and how it is done if so. Often, you don't have to think about it: it just works how you would intuitively expect. Such a type coercion is called *implicit*. For example, it is rarely a problem to mix different numerical types in an expression:

```
my_int = 1
my_float = 1.0
c = my_int + my_float
type(c)
```

```
float
```

Here the value in `my_int` is implicitly converted to a float which is then added to the float in `my_float`, and the result stored in `c` is another float.

As mentioned earlier, there are situations where such a coercion will not work. If, for example, you evaluated the expression `my_int + my_string`, you would get a `TypeError`. In such cases, you can make an *explicit coercion* by giving the value you want to convert as an argument to the type you want to convert it to. If we wanted to coerce a value stored in the variable `x`, for example, we could write:

```
str(x)
int(x)
float(x)
complex(x)
bool(x)
```

As a concrete example, we could write the following:

```
my_int = 1
my_string = "1"
c = my_int + int(my_string)
type(c)
```

```
int
```

Here, Python was able to convert the value in `my_string` to an integer to then add to `my_int`.

Warning: explicit coercion is not “magic” and will not always work:

```
int("hello!")
```

```
ValueError: invalid literal for int() with base 10: 'hello!'
```

Here, Python has no idea how to convert `'hello!'` to an integer.

Explicit conversion is often important when one uses `input()`, which reads input and returns a value of type `str`. In some cases one does not want the result as a string, as in the following:

```
age = int(input("Your age: "))
added_age = age + 10 # would not work if age was of type str
print(added_age)
```

Warning: Before v3, Python's `input()` tried to automatically interpret the type of its answer for you. This led to many bugs and security problems!

2.3 Boolean expressions in Python

Earlier, we saw examples of typical arithmetic operations (+, -, *, /, ...) that result in a value whose type depends on the types of the input arguments. Boolean expressions are expressions that result in a bool, that is True or False. Such expressions are very useful when one wants to check whether some condition is true or false.

It is worth pointing out that *most* values in Python are regarded as *true*. We can see this by using type coercion to bool:

```
bool(3)
bool(-427.2)
bool('hello')
```

```
True
True
True
```

Values that are regarded as *false* are typical “empty” values:

```
bool(0)
bool(0.0)
bool('')
bool(None)
```

```
False
False
False
False
```

2.3.1 Comparison operators

These operators compare their operands to check if various relations hold between them:

- Equality: `x == y`
- Inequality: `x != y`
- Strictly greater than: `x > y`
- Strictly less than: `x < y`
- Greater than or equal: `x >= y`
- Less than or equal: `x <= y`

These work as one would expect for numbers (experiment!), but like many operators in Python, you can also compare values other than ints and floats. For example:

```
print('hello shark' < 'hello shark')
print('hello shark' < 'hello shork')
print('hello shark' < 'hello')
print('hello shark' < 'Hello shork')
```

```
False
True
False
False
```

Google and see how Python compares strings!

2.3.2 Logical operators

Python includes the logical operations and, or och not.

- Are *x and y* true?: *x and y*
x and y is True only if both *x* and *y* are True.
- Is *x or y* true?: *x or y*
x or y is True only if at least one of *x* and *y* is True.
- Negate a value: *not x*
not x is True if *x* is False and False otherwise.

These rules can be written out with truth tables. We can, for example, use logical operators to define an expression which is an exclusive or, xor, that is evaluates to true if exactly *one* of *x* or *y* is true:

```
x = True
y = False
xor_result = (not ((x and y) or (not x and not y)))
print(xor_result)
```

```
True
```

Python handles and and or specially if (both of) the operands are not of type bool: an and/or expression is not evaluated in this case to a bool. An and-expression returns the *second* operand if both operands are true, but returns the value of the first *false* operand if there is one. An or-expression returns the *first* value if the first operand is true, otherwise the value of the second operand. Some examples:

```
(42 and 'hello')
(42 and '')
(False and '')
(42.7 or 13)
('42' or None)
(False or 0)
```

```
'hello'
''
False
42.7
42
0
```

These evaluation rules can seem strange, but they have to do with how much of the expression needs to be evaluated to know the result of the whole expression. If the first operand to and is false, then it is not necessary to evaluate the second value; regardless of whether it evaluates to True, the result will be False. This time-saving technique is sometimes called “short circuiting”, or more generally “lazy evaluation” (because it can be seen as “laziness” to not evaluate everything). The purpose is of course to evaluate a program more quickly by not computing expressions that will not be needed.

2.4 Assignment operators

We have already seen that `=` is used for assignment. A convenient functionality in Python is the ability to combine these with arithmetic operators to get so-called “assignment operators”:

```
x += 3    # x = x + 3
x -= 3    # x = x - 3
x *= 3    # x = x * 3
x /= 3    # x = x / 3
x %= 3    # x = x % 3
x //= 3   # x = x // 3
x **= 3   # x = x ** 3
```

2.5 Functions

As your program gets bigger and perhaps more complicated, you’ll quickly realize the need to give some structure to your source code. You can start to do so with the help of *functions*. We have already used several of Python’s built-in functions, for example `print`. Functions in programming have, as the name suggests, commonalities with mathematical functions: they take one or more arguments and give back a return value. In programming, however, a function does not *need* to take any arguments at all or to return a value, and it might do things *besides* return a value.

The point of functions is to:

- Organize code in easily handled and reusable pieces
- Avoid repetition of code
- Simplify advanced programs
- Increase *modularity*: a program divided up in small pieces is easier to understand and change than a big, monolithic program
- Make the program more readable:
 - Clarify dependencies in the code: “inputs” as arguments, the result after **return**. This is reminiscent of a function in mathematics.
 - Dividing code up makes it easier to document.
 - Dividing code up makes it easier to see that it does what it should (for example by testing, or even just by looking at it).

Functions are therefore a very important tool for writing good code.

Before you can use a function, you have to *define* it. This means using the reserved keyword **def** to give the function a name (identifier) and specify its arguments. Below is an example of a simple function `f` that takes a value `x` as an argument and adds the value to itself:

```
def f(x):
    return x + x
```

This function would be written in mathematics as $f(x) = x + x$. Once you have successfully defined a function, it can be called in the following way:

```
f(2.0)
```

```
4.0
```

Notice that Python is not concerned about what type the arguments have, as long as all the operators

and expressions in the function definition can be carried out. Above we applied `f` to a float, but we can try other types:

```
f(2)
f('hello')
```

```
4
'hellohello'
```

Just like mathematical functions, functions in Python can have multiple arguments:

```
def long_name(course_code, course_name):
    lname = course_code + course_name
    return lname
```

What is the result of `long_name('DA', '7066')`?

Functions do not need to take any arguments at all:

```
def show_user_menu():
    print('Menu:')
    print('1. Add user')
    print('2. List users')
    print('3. Quit')
```

and can call other functions themselves:

```
def user_choice():
    show_user_menu()
    return input('Which assignment do you want to do? ')
```

2.5.1 About functions

The examples above introduced several new syntactic concepts in Python. A function has a definition (also called its “body”) and a section (or “header”).

- The header consists of the function name and a list of parameters (the function’s arguments).
 - Syntax: `def functionname(argument1, argument2, ...):`
- The word parameter has two meanings.
 - Variables in the header are called *formal parameters*. Formal parameters are placeholders for the values that will be supplied when a function is called.
 - The values one gives when one calls a function are called *actual parameters*.
- The body has a documentation string (optional) followed by code, and is *indented*.

Indentation is important in Python! Python is an “indentation-sensitive” language, which is to say that incorrect indentation is a syntax error. Indentation is what specifies *code blocks* that belong to a function body. You can use any size of indentation you want in Python (for example, two spaces or a tab), as long as you are consistent, but the *standard* indentation is four spaces.

That indentation should be four spaces is specified in Python’s style guide, known as [PEP-8](#), which also contains lots of other good information on how you can write good Python code.

Most modern program editors will insert four spaces when you press the Tab key. Be warned that older or more primitive editors might not handle the Tab key in a good way. The way the results look might

depend on how your editor is configured, so that your code looks different when opened in different editors or on different computers.

The function body often ends with the reserved keyword **return**. If **return** is missing, then the value **None** is returned. It is good to know that **return** can appear several times in one function, but if any **return** is executed then the function will always finish and return the value in the executed statement.

It is important to understand the difference between functions that return something and functions that only print something out. Consider the following code snippet:

```
def f1(x):  
    return x + x  
  
def f2(x):  
    print(x + x)
```

The difference is that *f1 returns* the value in $x + x$ while *f2 only prints* the value. If we now try to print the result of *f1(2)*, it works as expected:

```
print(f1(2))
```

```
4
```

but if we try to print the result of *f2(2)*, we get the following:

```
print(f2(2))
```

```
4
```

```
None
```

The reason is that 4 is first printed in the course of executing the function *f2*, but later when we print the result of *f2* we get **None**, which is in fact what *f2* returns! It is very important that if you want to write a function that *returns* something, you must use **return** and not **print**.

2.5.2 Multiple return values

What do you do if you want to return multiple values? There are several bad ways of doing it, but one good one: multiple return values.

```
def integer_div(nominator, denominator):  
    q = nominator // denominator  
    r = nominator % denominator  
    return q, r
```

You can then call the function like so:

```
quotient, remainder = integer_div(17, 10)
```

2.5.3 Default values

It is common to write functions with parameters that are almost always set to the same value. Python lets you give such common values explicitly; these are called *default values*.

```
def f(x=1):  
    return x + x
```

```
print(f())  
print(f(3))
```

```
2  
6
```

In the first printed line, we can see that `f` was called with `x=1`, even though we did not give it as an argument. When we call `f(3)` on the next line, `x` is set to 3 as normal.

2.5.4 Keyword parameters

Python also lets you specify parameters by names when you call a function. For example, `integer_div(nominator=17, denominator=10)` has the same effect as `integer_div(17, 10)`.

What happens when we try:

```
integer_div(denominator=10, nominator=17)
```

Python connects the identifiers among the formal parameters to the identifiers in the arguments, so the above results in precisely the same output.

An example of a function we have seen that has several keyword parameters is `print`:

- `sep=' '` — use a space as separator
- `end='\n'` — “backslash-n” means newline
- `file=sys.stdout` — send output to the terminal

Exempel:

```
print('hello', 'you', sep='\n')
```

```
hello  
you
```

To see more information, type `help(print)` or search the Python online documentation.

2.5.5 Documentation of functions

There are two ways to document in Python:

- Documentation strings
 - Put at the top of a function body in between either `"""` or `'''`
 - Primary documentation method.
 - Tied to the function. Try `help(print)` in the Python interpreter.
- Comments
 - Used to explain calculation steps, typically single lines or small blocks of code.
 - Comments are for understanding *how* a function works. You should be able to *use* a function without reading the comments.

Documentation strings are described in [PEP-257](#):

The docstring [...] should summarize its behavior and document its arguments, return value(s), side effects, exceptions raised, and restrictions on when it can be called (all if

applicable). Optional arguments should be indicated. It should be documented whether keyword arguments are part of the interface.

An example of a single-line documentation string:

```
def square(a):
    '''Returns argument a squared.'''
    return a*a

print (square.__doc__)

help(square)
```

An example of a multi-line documentation string:

```
def square(x):
    """Description of square function

    Parameters:
    x (int): input number

    Returns:
    int: Square of x
    """

    return x*x

print(square.__doc__)

help(square)
```

A general principle for documentation is that you should explain what is not obvious from the identifier. If a function is called `compute_integral`, it is not necessary for the documentation string to say the same thing. Instead, it might be good to write which method is used, what assumptions are made about the parameters, and what is guaranteed about the result.

2.6 Global and local variables

Variables in Python can be either *global* or *local*. These properties indicate where a variable is “visible” to code and where the variable can be modified. In the following example, `x` is a global variable, as it was defined on the top level (not within a function body).

```
x = "global"

def foo():
    print("x printed from inside foo:", x)

foo()
print("x printed from outside foo:", x)
```

The function cannot modify `x`: the value that `x` holds can only be read.

```
x = 42

def foo():
    x = x * 2
    print("x printed from inside foo:", x)
foo()
```

UnboundLocalError: local variable 'x' referenced before assignment

To be able to change the value in x, we need to first tell the function that it can change this global variable. We do this with the reserved keyword **global**.

```
x = 42

def foo():
    global x
    x = x * 2
    print(x)
foo()
```

Warning: global variables that are changed by functions in this way can easily lead to “spaghetti code”. It can become extremely hard to keep track of where the value of x was changed! So be very careful if you use global variables in this way. There are, however, scenarios where they are very useful, for example as global counters. It is therefore very good to give global variables informative names, so that you do not change one by mistake!

Local variables are variables that are defined inside of a function:

```
def foo():
    y = "local"
    print(y)

foo()
```

The variable y is not visible outside the function, which is why it is called a *local* variable:

```
def foo():
    y = "local"

foo()
print(y)
```

NameError: name 'y' is not defined

Where x is visible is called its *scope*. If x is defined outside of the function and we later write over it within the function, its value does not change outside the function:

```
x = 5

def foo():
    x = 10
    print("local x:", x)
```

```
foo()
print("global x:", x)
```

The same thing happens with function arguments:

```
x = 5

def foo(x):
    print("local x:", x)

foo(12)
print("global x:", x)
```

2.7 Conditional statements

One often wants to execute a certain code block only if some expression is true or condition is met. This is done with so-called *conditional statements*, or “if”-statements. In python, we write these with the keywords **if**, **elif** (which is short for *else if*), and **else**.

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
print('Okay, I guess you were not finished yet.')
```

Obs: note the double equals sign! There is a difference between *comparison* (==) and *assignment* (=).

The above example could also be written:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not finished yet.')
```

Note that the print-statement is indented by the same amount as quit. Just like for functions, it is the indented code block that is executed.

Indentation errors can be hard to see sometimes:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not...')
    print('.. quite finished yet.') # indentation error
```

One more example with **else**:

```
namn = input('What is your name? ')
if namn == 'Anders':
    print('Hello Anders!')
else:
```

```
print('Hello, whoever you are!')
```

You can use *nested* conditional statements, which is to say if-statements inside if-statements:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
else:
    if temp < 0:
        print('That is pretty cold...')
    else:
        if temp == 0:
            print('Zero')
        else:
            if temp < 30:
                print('That is a comfortable temperature')
            else:
                print('Wow! Super warm')
```

In the above case, the code is not particularly readable. A better alternative would be to use **elif**:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
elif temp < 0:
    print('That is pretty cold...')
elif temp == 0:
    print('Zero')
elif temp < 30:
    print('That is a comfortable temperature')
else:
    print('Wow! Super warm')
```

Readability is important! **elif** is strictly speaking unnecessary, as it can always be replaced with nested **if-else** statements, but it is very useful when it makes the code easier to read. This type of functionality is sometimes called *syntactic sugar*.

What is the difference in meaning between

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
if x > 42:
    print('Too high...')
else:
    print('Too low...')
```

and

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
```



```

elif x > 42:
    print('Too high...')
else:
    print('Too low...')

```

What happens if one enters 42?

Finally, we can make use of conditional statements to rewrite the xor-expression from [Logical operators](#) and define a function:

```

def xor(x, y):
    if x and y:
        return False
    elif not x and not y:
        return False
    else:
        return True

print(xor(True, True))
print(xor(True, False))
print(xor(False, True))
print(xor(False, False))

```

```

False
True
True
False

```

Which version is easier to read?

2.8 Exercises

- Which alternative/alternatives should you keep in mind when choosing identifiers for variables and functions?
 - Identifiers should be short
 - Identifiers should be descriptive
 - Identifiers should be unique within the function/module/program/script
 - An identifier should be written with capitals
 - None of the above
- Write code that read an integer from the user and prints “Odd” if it is an odd number and “Even” if the number is even.
- Make *truth tables* for the boolean operators and and or (i.e., replace the question marks with True or False in the table below).

x	y	x and y	x or y
True	True	?	?
True	False	?	?
False	True	?	?

x	y	x and y	x or y
False	False	?	?

4. Try, without running the code, to evaluate these four expressions for all possible values of x, y and z:

```
(not x) or (not y)
x and (y or z)
(x != z) and not y
x and z
```

Then run the code to verify your answers.

5. Write a Python function that corresponds to the mathematical function $f(x, y) = x/y$. We want the function to notify the user if a number other than 0 is divided by 0. In this case, the function should print out `The result is infinite...` but *not* return any explicit value. If `0/0` is called, the function should print `Indeterminate form...` and not return anything either.
6. Let:

```
x = True
y = False
```

Which expressions below evaluate to `True`? Why?

```
(x and y) or (not x and not y)
(x or y) and (not x or not y)
x or y or not x or not y
not(not x and not (x and y))
```

7. Define `nand`, `nor`, and `xnor` as Python functions.

Tip: https://en.wikipedia.org/wiki/Logic_gate#Symbols.

8. Test the functions `nand`, `nor`, and `xnor` with arguments that are *not* of the type `bool`. Are the results what you expected? If not, why?

Chapter 3

Lists, tuples, and iteration

This chapter deals with:

- Lists
- Tuples
- Iteration with **for** and **while** loops

Lists and tuples are important in programming because they let us collect several elements together. Iteration deals with automatic repetition and is a fundamental concept in so-called *imperative* programming. Two important constructs we will discuss in this chapter are **for** and **while** loops. Lists and loops are especially useful together. For example, you can write a program that reads in numbers from the user, saves them in a list, and calculates the average of the inputs.

3.1 Lists

Python has support for lists of elements with different types. These are written in square brackets (“[” and “]”), and the elements of the list are separated with commas:

```
mylist = ["hej", 2, "du"]
```

Python’s list can be compared with *array* types in other languages, but it is a bit less efficient. However, Python’s lists are easier to use. This is typical for scripting languages: data structures are simpler to use at the price of efficiency.

3.1.1 List indexing

You can access elements from a list by writing `mylist[i]` where `i` is an integer:

```
print(mylist[0])
print(mylist[1])
print(mylist[2])
```

```
hej
2
du
```

Important: lists are indexed beginning with 0!

You can also use negative values to access an element from the end of the list.

```
print(mylist[-1])
print(mylist[-2])
print(mylist[-3])
```

```
du
2
hej
```

If you use an index that is too large, you'll get an error message:

```
print(mylist[3])
```

```
IndexError: list index out of range
```

Question: what happens if you index with a negative value that is too large (for example `mylist[-4]`)?

One of the key features of lists are that you can add, modify, and remove elements from the list after it is created. This is called *mutability* in programming, i.e., lists are *mutable*. For example, you can add elements to the end of a list using `append`.

```
mylist.append(True)
print(mylist)
```

```
['hej', 2, 'du', True]
```

You can easily assign a new value to a particular index in a list.

```
mylist[1] = 5
print(mylist)
```

```
['hej', 5, 'du', True]
```

We can even combine this with indexing from the end of the list:

```
mylist[-2] *= 3
print(mylist)
```

```
['hej', 5, 'dududu', True]
```

You can access all elements at indices in the range `i < j` using `mylist[i:j]`. This is called *slicing*.

```
print(mylist[0:1])
print(mylist[0:2])
print(mylist[1:3])
print(mylist[1:4])
```

```
['hej']
['hej', 5]
[5, 'dududu']
[5, 'dududu', True]
```

You can change the step size in the slicing to `k` by using `mylist[i:j:k]`:

```
print(mylist[0:4:2])
print(mylist[1:4:2])
```

```
['hej', 'dududu']
[5, True]
```

3.1.2 List functions

You can add lists and multiply them with a number to repeat them:

```
print(mylist + mylist)
print(2 * mylist)
```

```
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]
```

The length of a list can be calculated with len:

```
print(len(mylist))
print(len(5 * mylist + mylist))
```

```
4
24
```

A very useful function is **in**, which tests if an element occurs in a list:

```
print(5 in mylist)
print(False in mylist)
```

```
True
False
```

Compare with $x \in S$ from mathematics, which says that x is in the set S . The function **in** also works on strings:

```
print("gg" in "eggs")
```

```
True
```

There is also **not in**, which corresponds to $x \notin S$ from mathematics:

```
print(5 not in mylist)
print(False not in mylist)
```

```
False
True
```

You can find the smallest and largest element of a list using min and max:

```
print(min([2,5,1,-2,100]))
print(max([2,5,1,-2,100]))
```

```
-2
100
```

You can find the first index at which an element occurs with the help of **index**:

```
print(mylist.index(True))
```

```
3
```

If you try to find the index of something that doesn't occur in the list, you'll get an error message:

```
print(mylist.index(42))
```

```
ValueError: 42 is not in list
```

You can count the number of times an element occurs in a list with `count`:

```
mylist.append(5)
print(mylist.count(5))
print(mylist.count(42))
```

```
2
```

```
0
```

We can remove the element at a specific index or a whole slice with `del`:

```
print(mylist)
del mylist[2]
print(mylist)
del mylist[0:2]
print(mylist)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
[True, 5]
```

Writing `del s[i:j]` has the same effect as writing `s[i:j] = []`. You can even write `del s[i:j:k]` to use a different step size.

We can also remove the element at index `i` and then return it using `pop`:

```
mylist = ['hej', 5, 'dududu', True, 5]
print(mylist)
x = mylist.pop(2)
print(mylist)
print(x)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
dududu
```

This also works with negative values (that is, with indexing from the end). If no argument is given to `pop`, it behaves like `pop(-1)`: the last element is removed.

```
x = mylist.pop()
print(mylist)
print(x)
```

```
['hej', 5, True]
5
```

You can insert an element `x` at a given index `i` using `insert(i,x)`:

```
mylist.insert(1,"du")
print(mylist)
```

```
['hej', 'du', 5, True]
```

This has the same effect as writing `s[i:i] = [x]`.

To remove the first element that is equal to `x`, you can use `remove(x)`:

```
mylist.append(5)
print(mylist)
mylist.remove(5)
print(mylist)
```

```
['hej', 'du', 5, True, 5]
['hej', 'du', True, 5]
```

Finally, you can use `reverse` to reverse a list:

```
mylist.reverse()
print(mylist)
```

```
[5, True, 'du', 'hej']
```

3.1.2.1 Summary of useful list functions

- `mylist.append(x)`: add `x` to the end of `mylist`.
- `len(mylist)`: calculate the length of `mylist`.
- `x in mylist`: test if `x` occurs in `mylist` (returns a bool).
- `x not in mylist`: test if `x` doesn't occur in `mylist` (returns a bool).
- `min(mylist)` and `max(mylist)`: find the smallest, respectively largest element of `mylist`.
- `mylist.index(x)`: calculate the first index where `x` occurs in `mylist`.
- `mylist.count(x)`: count how many times `x` occurs in `mylist`.
- `del mylist[i]`: remove the element at index `i` from `mylist` (without returning it).
- `mylist.pop(i)`: remove the element at index `i` from `mylist` (and return it).
- `mylist.insert(i,x)`: insert `x` at index `i` in `mylist`.
- `mylist.remove(x)`: remove the first occurrence of `x` from `mylist`.
- `mylist.reverse()`: reverse the order of elements in `mylist`.

For more list functions and details on lists, see: <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

<https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>

<https://docs.python.org/3/library/stdtypes.html#lists>

3.1.3 A practical function for building lists: range

One way to create lists of integers is to use `range`:

```
print(list(range(1,4)))
print(list(range(3)))      # same as range(0,3)
print(list(range(4,2)))
```

```
print(list(range(0,10,2)))
print(list(range(5,0,-1))) # iterate backwards
```

```
[1, 2, 3]
[0, 1, 2]
[]
[0, 2, 4, 6, 8]
[5, 4, 3, 2, 1]
```

Obs: we have to convert the result to a list with the help of `list(...)` because `range` does not directly return a list, but a “promise” of a list. That is, `range` returns an object that creates a list when you need it. This behavior is called *lazy evaluation* and is convenient when you do not need to create the whole list at once. This has the advantage that the whole list need not be saved in the computer’s memory all at once, which could be slow for large lists. More on this later.

If you try to print `range(0,10)` without converting to a list, you’ll get the following result:

```
print(range(0,10))
```

```
range(0, 10)
```

3.1.4 Characteristics of lists

Lists have many strengths, such as:

1. Simple and efficient to iterate over (more on this in the next section).
2. Simple to extend.
3. Immediate access to any particular index.
4. List elements can have different types.

Characteristic 3 is specific to Python. Other languages, for example C and Java, differentiate between *arrays* and *lists*. An array is like a vector and has a fixed length. In these languages, lists are instead a special data structure in which the elements are “linked” so that one can only immediately access the first (and possibly the last) element. Thus, one cannot automatically get immediate access to any element. In Python, though, one has direct access to all elements by list indexing, making these lists somewhat reminiscent of arrays.

A weakness of lists is that it is inefficient to *search* in them: the operator `in` becomes slow with large lists. Another weakness is that unexpected things can happen if one has a list containing elements of different types and tries to find the largest or smallest element (see exercise 2).

3.2 Tuples

Tuples are similar to lists, but they are *immutable*, so we cannot add, modify or remove elements from a tuple. One might think that this is a limitation. But there are at least two cases where immutable datatypes are desired:

1. When used in dictionaries (which we will discuss in chapter 4)
2. When we want to guarantee that the datatype cannot be changed, e.g., by accident.

Point 2 is especially useful when programs grow large (thousands of lines) and complex to manage. We have seen tuples before, even if we haven’t explicitly created variables of tuple type: multiple return

values (Chapter 2) from functions are in fact tuples of multiple elements. The syntax for tuples is very similar to that for lists, the difference being that we use `()` to create a tuple instead of `[]`.

```
mytuple = (3, 3, 4, 5)
myothertuple = 3, 3, 4, 5 # equivalent syntax
print(mytuple)
print(mytuple == myothertuple)
print(mytuple[0])
print(mytuple[3:1:-1])
print(len(mytuple))
print(3 in mytuple)

# function returning multiple values
def integer_div(nominator, denominator):
    q = nominator // denominator
    r = nominator % denominator
    return q, r

divs = integer_div(17, 10)
print(type(divs))

# tuples are immutable
mytuple[0] = 42

(3, 3, 4, 5)
True
3
(5, 4)
4
True
<class 'tuple'>
TypeError: 'tuple' object does not support item assignment
```

You can also slice a tuple just like a list.

```
mytuple = (3, 3, 4, 5)
print(mytuple[1:3])
```

```
(3, 4)
```

Just like lists, tuples can contain elements of different types, and we can nest them:

```
t = (12345, 54321, 'hello!')
mytuple = (t, (1, 2, 3, 4, 5))
print(mytuple)
```

Tuples can even contain *mutable* objects.

```
t2 = ([1, 2, 3], [3, 2, 1])
print(t2)
```

But this violates the nice property of imutability and then, they *cannot* be used as a key in a dictionary (more about this in Chapter 4).

3.3 for loops

for is used to iterate over enumerations of different kinds, in particular lists. These enumerations are called *sequences* in Python. Even strings are sequences (of characters).

```
for x in [1, 2, 3]:  
    print(x)
```

```
1  
2  
3
```

Obs: note that the `print` statement is indented! Everything that comes after **for** and is indented (by the same amount of spaces) will be run in the loop. As we saw in the previous chapter, Python is indentation-sensitive: it is important that you give all lines in a code block the same indentation, and this applies to **for** loops as well. This differentiates Python from languages such as C or Java, where one instead groups code blocks within curly braces (“{” and “}”) and lines end with a semicolon “;”. If something is indented incorrectly, Python will show the following error message:

IndentationError: expected an indented block

A code block can contain several instructions:

```
s = "A number: "  
  
for x in [0,1,2]:  
    print(x)  
    s += str(x)  
  
print(s)
```

```
0  
1  
2  
A number: 012
```

We can also iterate over a list stored in a variable:

```
mylist = [112, 857, "hej"]  
  
for x in mylist:  
    print(x)
```

```
112  
857  
hej
```

As strings are also sequences, one can easily iterate over the letters in a string:

```
for c in 'DA2004':  
    print(2*c)
```

```
DD  
AA
```

```
22
00
00
44
```

3.3.1 A function that extracts the consonants from a string

Let's now implement a slightly more complicated function that extracts the consonants from a string. Before we begin writing code, we should think about how the function should work. Given a string *s*, it should iterate over all the letters in *s* and test if they are consonants. For each letter, if it is a consonant, then it should be added to a string with all the consonants we have found in *s*; otherwise, nothing should happen. This can be implemented as follows:

```
def consonants(s):
    cs = "BCDFGHJKLMNPQRSTVWXZbcdfghjklmnpqrstvwxyz"

    out = ""

    for c in s:
        if c in cs:
            out += c

    return out

print(consonants("Hey Anders!"))
```

```
Hndrs
```

Note how we defined a variable *out* which we used to keep track of the consonants we found in *s*. This is a very common way of programming and one we will see more of in the course.

3.3.2 Dividing up a string into words: `split`

We'll now try writing a function that takes a string and divides it up into pieces wherever it encounters a blank space. That is, we'll write a function `split(s)` that takes in a string *s* and returns all the words in the string. We can do this using two strings, one with the words we have found so far and one with all the characters in the word we are building up. For every character in *s*, we test if it is a space or not. If it is, we add our current word into the list; otherwise, we add the new character to our current word. Then we move on to the next character.

```
def split(s):
    out = []
    term = ""

    for c in s:
        if c == " ":
            out.append(term)
            term = ""
        else:
            term += c
```

```

    # add the final word unless it is empty
    if term != "":
        out.append(term)

    return out

print(split("Hey Anders! How are you?"))

```

```
['Hey', 'Anders!', 'How', 'are', 'you?']
```

Suppose now that we want to generalize this function to handle many different separators, with the blank space as the default value. Then we can write the following instead:

```

def split(s, sep = " "):
    res = []
    term = ""
    for c in s:
        if c in sep:
            res.append(term)
            term = ""
        else:
            term += c

    if term != "":
        res.append(term)

    return res

print(split("Hey Anders! How are you?"))
print(split("Hey Anders! How are you?", "!"))

```

```
['Hey', 'Anders!', 'How', 'are', 'you?']
['Hey Anders', ' How are you?']
```

Now we can call the function with one *or* two arguments.

Obs: There is already a version of `split` built into Python!

3.3.3 Control flow: `pass`, `continue`, `break` and `return`

We can control and stop iteration with `pass`, `continue`, `break` and `return`.

The simplest of these is `pass`, which does nothing.

```

for i in range(4):
    if i == 1:
        pass
        print("passed 1")
    else:
        print(i)

```

```
0
passed 1
2
3
```

Obs: `pass` can seem useless at first, as it doesn't do anything, but a common use is as a placeholder when you've started outlining the structure of your code but haven't filled in the details yet. For example, you can define a function whose body is just `pass` or have an `if` statement where you haven't filled in either case yet. In this way, you can test all the other code in the program now, then replace the `passes` when you are sure that everything else works.

A `continue` instruction ends the current iteration of the loop and moves on to the next:

```
for i in range(4):
    if i == 1:
        continue
    print("passed 1")
else:
    print(i)
```

```
0
2
3
```

A `break` instruction ends the loop entirely:

```
for i in range(4):
    if i == 2:
        break
    else:
        print(i)
```

```
0
1
```

Finally, you can use `return` in a loop to return from a function, as always. For example:

```
def first_even_number(l):
    for number in l:
        if number % 2 == 0:
            return number

print(first_even_number([1,45,24,9]))
```

```
24
```

3.3.4 Python's `for` is different

It is good to be aware that that Python's `for` loop is more general than in most other languages. For example, `for` in other languages can often only iterate over integer values. Python's version is more "modern": it works for any data structure that supports iteration. It is possible, and not so hard, to write your own iterable datatypes for special purposes. So far we have seen that you can iterate over lists and strings; we will see more examples in the future.

3.3.5 A common problem with lists in Python: mutability

A common mistake made with lists in Python, one we will see several times in the course, is accidentally changing a list while iterating over it. This is a classic source of bugs, both for beginners and more experienced programmers.

Suppose we have the following program, which we expect to take in a list and copy each word in it with more than 6 letters to the front of the list.

```
animals = ["cat", "dog", "tortoise", "rabbit"]

for x in animals:
    if len(x) > 6:
        animals.insert(0, x)

print(animals)
```

But if we run this code, Python just loops forever! To stop the loop, you can press CTRL-C. The problem is that we changed the list `animals` at the same time that we looped over it. There is a logic to Python's behavior in such a case, but it is difficult to follow and rarely matches your intentions.

To avoid such problems, one should instead iterate over a *copy* of the list. We can do this by using `.copy()`:

```
animals = ["cat", "dog", "tortoise", "rabbit"]

for x in animals.copy():
    if len(x) > 6:
        animals.insert(0, x)

print(animals)
```

```
['tortoise', 'cat', 'dog', 'tortoise', 'rabbit']
```

Here we use `animals.copy()` instead of `animals` so that we get a copy of the list and avoid the endless loop.

This kind of problem often comes up with one programs with lists without being careful to make copies in the right places. It can lead to very mysterious bugs like the infinite loop above. What makes these issues come up is that Python's lists are *mutable* and many list operations work by *mutating* lists. We will return to mutability later in the course, but for the moment we just need to remember that we may need to use `.copy()` on a list variable to avoid accidentally changing it when we don't want to.

3.4 while loops

Sometimes there is no obvious structure to iterate over, but we still want to run a code block several times. For this, we have **while** loops. A **while** loop lets us run a block of code until some boolean condition is satisfied. Consider the following example:

```
i = 10

while i > 0:
    print(i)
```

```
i -= 1  
  
print("Done!")
```

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
Done!
```

A common use for **while** loops is to loop until the code gets proper input from a user. The program below will read in input from the user until they enter exit:

```
loop = True  
  
while loop:  
    print("Write 'exit' to quit")  
    answer = input("Write something: ")  
    if answer == "exit":  
        print("Goodbye!")  
        loop = False  
    else:  
        print("Let us keep going!")
```

```
Write 'exit' to quit  
Write something: hey  
Let us keep going!  
Write 'exit' to quit  
Write something: ho  
Let us keep going!  
Write 'exit' to quit  
Write something: exit  
Goodbye!
```

3.4.1 while loops over lists

Like many other datatypes, lists can be implicitly coerced to booleans. The empty list corresponds to False, while every other list corresponds to True.

```
list = []  
  
if list:  
    print("Not empty!")  
else:
```

```

    print("Empty!")

list2 = ["Hey"]

if list2:
    print("Not empty!")
else:
    print("Empty!")

```

```

Empty!
Not empty!

```

This capacity enables the following *idiom*, where one loops over a list and removes its elements:

```

mylist = ["Ho", "ho", "ho!"]

while mylist:
    x = mylist.pop()
    print(x)

print(mylist)

```

```

ho!
ho
Ho
[]

```

When the loop is over, the list is empty, so one need not compare the list with `[]` explicitly. This is also an example where we change a list while looping over it.

3.4.2 Remove spaces from the end of a string

With the help of a `while` loop and slicing, we can remove the blank spaces from the end of a string:

```

def remove_trailing_space(s):
    i = len(s)
    while i > 0 and s[i-1] == " ":
        i -= 1
    return s[0:i]

print(remove_trailing_space("SU "))

```

```

SU

```

We begin with a counter `i` which is set to the length of the list. Then we count the number of characters that are blank spaces starting from the end of list, then return a slice of the original list that leaves out these blank spaces.

3.4.3 Greatest common divisor: GCD

The *greatest common divisor* (GCD) of two numbers is the largest integer that divides both of them. For example, 3 is the GCD of 15 and 12.

Euclid's method for calculating the GCD of two numbers a and b works as follows:

- Suppose $a \geq b$,
- take r to be the remainder of the integer division $a // b$,
- if $r == 0$, then $\text{GCD}(a, b) = b$,
- otherwise, we can use the fact that $\text{GCD}(a, b) = \text{GCD}(b, r)$.

Note that this is an iteration with a conditional, but without any clear structure to iterate over. It is therefore a good case for a **while** loop:

```
def GCD(a,b):
    if b > a:
        print("a must be bigger than b in GCD(a,b)")
        return None

    r = a % b
    while r != 0:
        a = b
        b = r
        r = a % b
    return b

print("GCD: " + str(GCD(15,12)) + "\n")
print("GCD: " + str(GCD(12,15)))
```

GCD: 3

b must be bigger than a in GCD(a,b)
GCD: None

3.5 Exercises

1. What is the result of `[1,2] + ['a', 'b']`?
2. What happens if we use `min/max` on lists with elements of different types?
3. Complete the function

```
def vowels(s):
    vs = "AEIOUYaeiouy"
    pass
```

that takes in a string s and returns its vowels.

4. Modify the function in exercise 3 so that it instead returns the consonants in s , but do so without changing the string `vs`. Then write a function `vowels_or_consonants` taking an extra parameter that determines whether vowels or consonants should be returned. The parameter should have a default value so that vowels are returned if the programmer calls the function with only one argument.
5. What happens if you leave out the code block beginning with `if term != ""` in the function `split(s)` above? Find an example of a string s such that the output is incorrect if it is missing.

Furthermore, how does `split` behave on strings with several spaces in a row? Write a correct version of the function.

6. Test the following code snippet:

```
list = [1,2,3]
list2 = list
list2.reverse()
print(list)
```

Does it work as you expected? Why not? How can you fix it?

7. Rewrite the code

```
mylist = ["Ho", "ho", "ho!"]

while mylist:
    x = mylist.pop()
    print(x)

print(mylist)
```

so that the list `mylist` is not empty at the end.

Tip: use `.copy()`. How can you do it without using `copy`?

8. Try using **`continue`**, **`break`** and **`pass`** in the **`while`** loop below. What happens?

```
n = 10
while n > 0:
    n -= 1
    if n == 7:
        # test continue, break or pass here
    print(n)
```

9. Write a **`while`** loop that reads in numbers from the user and saves them in a list. It should continue to read in until the user enters 0; then the program should end the loop and print out the list.

10. Write the function

```
def naiveGCD(a,b):
    pass
```

that calculates the GCD of `a` and `b` by testing all numbers between 1 and `min(a,b)` to see which is the biggest number that divides both.

11. Write the function `passcode()` that reads in one digit (1–9) at a time from the user. After each digit is read, the function should test if the last four digits entered matches the correct code. If the correct code is entered, the function should print `Door unlocked!` and return `True`. Choose the correct passcode yourself, for example 1337.

Tip: What loop should we use?

Chapter 4

Dictionaries, file handling, more on loops

A “data structure” is a type for storing data. There are many kinds of data structures; we have seen a couple in the course already, among them lists. In this chapter, we will introduce dictionaries, which are like lists that can be indexed by types other than integers. Another way to save data is to write it to a file on the computer. We will also see how to do that in this chapter. Finally, we will look at more we can do with loops.

4.1 Dictionaries

The dictionary is a data structure that has become a “workhorse” for scripting languages. They are easy to use and useful in many situations. Dictionaries are much like lists, but instead of being indexed by integers, they are indexed by *keys*. You can think of a dictionary as a collection of *key-value pairs*. In Python, the type of dictionaries is called `dict`.

There are two typical ways to create a dictionary in Python. For example, you can construct a dictionary that has keys 'DA7066' and 'MM2001' with respective values 7.5 and 30 like so:

```
hp1 = dict(DA7066=7.5, MM2001=30)
hp2 = {'DA7066': 7.5, 'MM2001': 30}
```

Both methods give the same result, and `hp1 == hp2`. Note that the first uses the same syntax as calling a function with keyword parameters DA7066 and MM2001, while the second uses a *string* for each key with its value separated by a colon.

To access a value, we use `[]`-syntax with a key:

```
print('Number of ECTS points for DA7066: ' + str(hp1['DA7066']))
```

```
Number of ECTS points for DA7066: 7.5
```

Note that the key argument can be an expression. That expression is first evaluated, and the result is the key then looked up in the dictionary. The same principle applies to the `{}` syntax for creating dictionaries. In both `hp1` and `hp2`, the keys have type `str`. An example that illustrates this point:

```

DA7066 = 33
MM2001 = 'hello'
hp3 = {DA7066: 7.5, MM2001: 30}
print('Number of ECTS points for DA7066: ' + str(hp3[DA7066]))
print('Number of ECTS points for DA7066: ' + str(hp3[33]))
print('Number of ECTS points for MM2001: ' + str(hp3[MM2001]))
print('Number of ECTS points for MM2001: ' + str(hp3['hello']))

```

```

Number of ECTS points for DA7066: 7.5
Number of ECTS points for DA7066: 7.5
Number of ECTS points for MM2001: 30
Number of ECTS points for MM2001: 30

```

Here a dictionary is created with keys 33 and 'hello', the results of evaluating the expressions DA7066 and 'hello' respectively. Likewise, using either DA7066 or 33 as an argument to a lookup has the same result. If you tried to use a *string* of value 'MM2001', on the other hand, you would get an error, as that key is not in the dictionary:

```
print('Number of ECTS points for MM2001: ' + str(hp3['MM2001']))
```

```
KeyError: 'MM2001'
```

In Python, all objects that cannot be changed can be used as keys. Dictionaries are implemented with a technique called hash tables, which involves converting the keys to integers (by “hashing” them, which is to say chopping up and blending their bytes into integers) used for lookups. If the key can change, be “mutated”, then there is a risk of losing access to entries you created. In general, the rule is that *immutable* values, for example strings, numbers, and tuples, can be used as keys. Mutable types like lists are excluded.

Recall the example with a tuple containing a mutable data type from previous chapter:

```

t = (12345, 54321, 'hello!') # immutable
t2 = ([1, 2, 3], [3, 2, 1]) # mutable
print(t2)

```

```

d = {t: 'hello'} # works
d2 = {t2: 'goodbye'} # TypeError: unhashable type: 'list'

```

When you use dict to create a dictionary, the keyword parameters are always read as strings.

For more detailed documentation of dictionaries in Python, see:

<https://docs.python.org/3/tutorial/datastructures.html#dictionaries>

<https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

4.1.1 Important characteristics

- A lookup is cheap (that is, fast).
- It is easy to change the contents.
- Dictionaries are easy to iterate over (but you cannot add new entries to a dictionary during iteration).

4.1.2 Other names

- Hash table
- Hash map

In “traditional” languages (C, C++, Java, etc.), dictionaries do not play such a central role as in scripting languages, which make dictionaries especially easy to use.

4.1.3 More examples

An empty dictionary can be written as {}, which is syntactic sugar for dict().

```
empty = {}  
print(empty)  
print(dict())
```

```
{}  
{}
```

We can also create dictionaries in the following way:

```
test1 = {'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
test2 = dict([('hej', 4139), ('hopp', 4127), ('foo', 4098)])  
  
print(test1)  
print(test2)
```

```
{'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
{'hej': 4139, 'hopp': 4127, 'foo': 4098}
```

Note that the keys in a dictionary need not all have the same type!

We can update dictionaries in a way similar to lists, adding or updating key-value pairs with [-] and deleting them with **del**:

```
hp = {}  
hp['DA7066'] = 7  
print(hp['DA7066'])  
  
hp['DA7066'] += 0.5  
print(hp['DA7066'])  
  
hp['MM2001'] = 30  
hp['UG1001'] = 1  
print(hp)  
  
del hp['UG1001']  
print(hp)
```

```
7  
7.5  
{'DA7066': 7.5, 'MM2001': 30, 'UG1001': 1}  
{'DA7066': 7.5, 'MM2001': 30}
```

4.1.4 Iterating over dictionaries

We can use `for` to iterate over keys:

```
for key in hp:  
    print(key)
```

```
DA7066  
MM2001
```

We can also be more explicit by using `.keys()`:

```
for key in hp.keys():  
    print(key)
```

```
DA7066  
MM2001
```

or iterate over values instead using `.values()`:

```
for val in hp.values():  
    print(val)
```

```
7.5  
30
```

We can also extract all key-value pairs with `.items()`:

```
for key, val in hp.items():  
    print('Number of ECTS points for ' + key + ': ' + str(val))
```

```
Number of ECTS points for DA7066: 7.5  
Number of ECTS points for MM2001: 30
```

All of these functions return iterable objects with elements consisting of keys, values, or key-value pairs.

4.1.5 Functions for dictionaries

We can manipulate dictionaries with functions similar to those we had for lists:

```
print(len(hp))  
hp['foo'] = 20.0  
print(len(hp))
```

```
print('DA7066' in hp)  
print('20.0' in hp)
```

```
2  
3  
True  
False
```

One more useful function is `.get()`, which takes two arguments: a key and a value to be returned if the key is *not* in the dictionary. If the key is found, its associated value is returned.

```
print(hp.get('DA7066', 42)) # key exists
print(hp.get('bar', 42)) # key does not exist
```

```
7.5
42
```

4.2 File handling

Files are data stored on the hard disk: text files, pictures, video, and so on. We access files by interacting with the operating system. This can be done directly from Python.

4.2.1 Reading from files

To read from a file, for example some `data.txt`, we first create a “handle”. We do this using the function `open()`, which takes two required arguments: the name of the file as a string and a string that specifies if the file will be readable, writable, etc.—see the help/documentation string for more information.

```
h = open('data.txt', 'r') # 'r' for "read"
```

To read in all of the file’s contents, we then write:

```
h.read() # whole file in string
```

To instead get a list of strings where each list element is a line from the file:

```
h.readlines() # whole file as list of strings
```

In many cases, these functions work poorly with large files. Problems arise when we read in a file that is larger than the computer’s RAM.

To only read a single line from `h`, we use:

```
h.readline()
```

Note that the contents of a file are read through a handle *one time*: if we have already used `read` or `readlines` with `h`, `readline` will not do anything, as the handle/file pointer has already come to the end of the file.

4.2.2 Writing to files

If we want to write to a file `out_data.txt`, we must again first create a handle, this time with `w` for *write*.

```
output = open('out_data.txt', "w")
```

This either *creates* or *empties* `out_data.txt`. **Warning:** Be aware that everything in the file will disappear if it already exists! To now write to `output` (that is, the file `out_data.txt`), we use `write`:

```
output.write("This is my first line")
output.write("\n") # start a new line
output.write("This is my second line")
```

4.2.3 Closing files

When we are finished with a file, we should close it:

```
h.close()
output.close()
```

It is important to close files:

- The number of open files is limited in an operating system— don't be wasteful with them!
- Another program might want to write to the file—close your handle other programs don't need to wait!
- You can lose data if you don't close the file.

When a program finishes or is terminated, all its handles are closed automatically.

4.2.4 An example

Consider a file `people.txt` with comma-separated information: last name, first name, birth year.

```
Svensson,Lisa,1815
Olsson,Erik,1901
Mörtberg,Anders,1986
```

The following program creates a file `modern.txt` containing the full name of each person from the 1900's:

```
h_in = open('people.txt', 'r')
h_out = open('modern.txt', 'w')
for n in h_in:    # loop through the file, row by row
    lname, fname, year = n.split(',')
    if int(year) > 1900:
        h_out.write(fname + ' ' + lname + '\n')
h_out.close()
h_in.close()
```

If we now open the file `modern.txt`, we find it contains

```
Erik Olsson
Anders Mörtberg
```

4.2.5 Important idiom: with

Here we have an alternative notation to open/close:

```
with open('ut.txt', 'w') as out:
    out.write("hej hopp")
```

This little example corresponds to

```
out = open('ut.txt', 'w')
out.write("hej hopp")
out.close()
```

It is good practice to use 'with'—this is idiomatic Python. Some advantages:

- Easy to read
- Encourages good code structure

- Considered to simplify error handling
- The file is closed when execution leaves the `with`-block

In Python, `with` is called a *context manager*. It can also be used with things other than files.

The example above can be rewritten using `with` as follows:

```
with open('people.txt', 'r') as h_in, open('modern.txt', 'w') as h_out:
    for n in h_in:
        lname, fname, year = n.split(',')
        if int(year) > 1900:
            h_out.write(fname + ' ' + lname + '\n')
```

4.2.6 What to keep in mind when handling files

- Reading and writing files is slow.
 - Avoid reading a file more than once.
 - Use files as temporary storage only if you must.
- Some files are big. Avoid reading in everything if you don't need it.
- Avoid hard-coding file paths.
- A good rule of thumb: read from `stdin` and write to `stdout`. This is Unix tradition.
- Temporary files should have unique and temporary names
 - Use the module `tempfile` (<https://docs.python.org/3/library/tempfile.html>) for safety and convenience!

4.3 More on loops: nesting

The syntax for a `for`-loop allows the indented “body” to contain one or more statements. We can naturally combine *multiple* loops:

```
for i in range(3):
    for j in range(3):
        print(i * j)
```

```
0
0
0
0
1
2
0
2
4
```

Note that we increase the indentation within each loop. Pay careful attention to the indentation, as it determines the structure of the program and how it will be executed! An example:

```
print("-----")
for i in range(1,4):
    for j in range(1,4):
        print(i, j, i * j, sep=" ")
    print("-----")
```

```

-----
1  1  1
1  2  2
1  3  3
-----
2  1  2
2  2  4
2  3  6
-----
3  1  3
3  2  6
3  3  9
-----

```

Note that `print("-----")` is only run once in each pass through the outer loop. What would happen if we instead had it indented to align with the previous line `print(i, j, i * j, sep=" ")`?

We can also restrict to only compute even products with the help of `if`-statements in loops. Note the indentation levels of the two different `if`-statements.

```

for i in range(1,8):
    if i % 2 == 1:
        continue
    for j in range(1,8):
        if j % 2 == 1:
            continue
        print(i * j)

```

```

4
8
12
8
16
24
12
24
36

```

4.4 Exercises

- Given the two lists below

```

a = ['a', 'b', 'c']
b = [1, 2, 3]

```

create a dictionary `{'a': 1, 'b': 2, 'c': 3}` using a loop.

Tip: There are several ways to do this. One way is to use `range` to get an index that you can then use to access elements of `a` and `b`.

- Write code that takes all pairs (x, y) of numbers between 2 and 20 such that $x \neq y$ and prints out their greatest common divisor (GCD).

Tip 1: Use nested loops and range.

Tip 2: Use the function GCD from the previous chapter.

The code should for example write out the following:

```
x y GCD
2 3 1
2 4 2
2 5 1
2 6 2
2 7 1
2 8 2
# etc...
```

3. In exercise 2, it is unnecessary to print the GCD of, for example, both $x, y = 2, 3$ and $x, y = 3, 2$. Rewrite the code so that x is always less than y in the printed lines.

Tip: In the inner for-loop, what happens if we iterate starting from the current value of x ?

4. In chapter 3, we worked with the function `vowels`. Use a dictionary that has the vowels in `vs` as keys and numbers as values (you can choose the values yourself). Modify the function so that it instead replaces each vowel with the corresponding number.
5. In chapter 3, we worked with the function `split` (given below). Use the structure of this function as a guide to help you write a function that takes a string and associates each word with a number representing where the word appears for the last time in the string. The function should return a dictionary with words as keys and numbers as values.

```
def split(s):
    out = []
    term = ""
    for c in s:
        if c == " ":
            out.append(term)
            term = ""
        else:
            term += c

    # add the final word unless it is empty
    if term != "":
        out.append(term)

    return out
```

The function should produce results like so:

```
print(annotate_word("hej du hej hej du"))

{'hej': 4, 'du': 5}
```

6. How can we easily get `annotate_word` from exercise 5 to instead return a dictionary with numbers as keys and words as values?

7. Modify `annotate_word` from exercise 5 so that not only the last occurrence but *all* occurrences are stored in the dictionary. What data structure should we have as *values* in our data structure?

Chapter 5

Programming in practice: algorithmic thinking, data representation, and debugging

5.1 Algorithmic Thinking

When learning programming, one of the significant challenges is to develop a deep understanding of how the computer works and how it processes instructions. The Python language is a high-level language, meaning that the technical details of the computer have been largely abstracted away. It can be helpful to be aware of certain technical details, for example, the section on data representation (below), but in Python, you can go far and essentially pretend that you are working in an abstract environment without much thought about how your computer actually works.

An *algorithm* is a step-by-step procedure or set of rules designed to solve a problem or perform a task. It can be thought of as a sequence of well-defined instructions that, when executed by the program, lead to a desired outcome. In order to be a good computer programmer, you cannot escape algorithmic thinking: understanding how to go from a problem description to a solution (i.e., a step-by-step procedure) in the form of code. This can be challenging because, in our daily lives, we are not so meticulous about describing how things are done. When you communicate with others about what needs to be done in a project or assignment, much is left unsaid. People share so much experience and underlying knowledge that not all details need to be communicated. You and your computer, on the other hand, do not share much experience; to get the computer to do what you want, you must explain yourself in the smallest detail.

5.1.1 Recipes are not Algorithms

Recipes are often used as an analogy for algorithms and are sometimes used to illustrate algorithmic thinking. While the analogy is a good initial explanation of what an algorithm is, it can be misleading because recipes rely on a lot of underlying knowledge. If you have taught cooking to a child, you've likely noticed that it requires more than just reading the recipe.

Take the task of making pancakes, for example. Here is a recipe (with the first part about ingredients removed) that most of us can follow without a problem:

1. Whisk the flour into half of the milk until smooth.
2. Whisk in the remaining milk, egg, and salt.
3. Melt butter in the frying pan and pour it into the batter.
4. Bake thin pancakes in the frying pan. About 1 dl of batter per layer is sufficient.

Even in the first step, many details are missing. Should I whisk the flour before it goes into the milk, or is it better to dump all the flour into the milk and then start whisking? What tool should be used for whisking? And where should we work? Let's try to improve the first step to make the recipe more algorithmic:

1. Mix wheat flour and milk like this:
 - a. Take out a bowl that can hold 2 liters and pour in half of the milk.
 - b. Take out a whisk.
 - c. Pour in the flour, one tablespoon at a time, and whisk for 20 seconds between each tablespoon.
2. Whisk in the remaining milk, egg, and salt.
3. Melt butter in a frying pan and pour it into the batter.
4. Bake thin pancakes in the frying pan. About 1 dl of batter per layer is sufficient.

What has been achieved is the removal of a judgment (what is a "smooth batter"?) and the level of detail has increased, hopefully eliminating some ambiguity. Could a seven-year-old follow step 1?

Steps 2 and 3 can be made more algorithmic in a similar way, but note that step 4 has some greater challenges. Tasks are presented in the wrong order (the batter must be poured into the pan to be baked!), and judgments are left to the reader. Instead of suggesting 1 dl of batter, we should carefully explain "pour batter until the entire pan is covered by a thin layer, but no more, or until the batter runs out." The last condition handles the fact that the batter rarely covers the pan's surface for the last pancake. We also need to explain that the pan should be hot (how hot?), a spatula is needed, the batter is baked until the underside becomes brownish, and then it is flipped and the other side is baked. And what does "brownish" really mean?

As you can see, there is a lot to write, and the level of detail can quickly become a bit frustrating, but it may be necessary to describe to a robot how to bake pancakes in a "traditional way."

5.1.2 *Top-down, bottom-up, and doing it right*

When we improved the pancake recipe, we started with a rather rough description of what needs to be done and refined the first point. This is a common approach, especially when unsure about the task, known as working top-down. The idea is to gradually break down a task into smaller parts until reaching a level where it's possible to formulate code to solve the subtask.

The opposite approach is working bottom-up, identifying useful code that can then be composed into a larger whole. Using the pancake example, one might say, "I know how to mix flour and milk, and that will be needed, so I started coding that subtask."

You don't have to choose between top-down and bottom-up; both approaches are useful, and the important thing is to work with the task. Don't expect to contemplate the problem, see the solution, and then write the necessary code. It can be like that sometimes, especially for an experienced programmer, but experimenting and searching for solutions is not wrong.

Remember that written code can and should be modified as needed. Beginners may be satisfied that their code works and sometimes avoid making changes. However, modifying code is often necessary, and putting off issues can lead to more challenging problems later.

It's easy to make a slightly inappropriate design choice, and when discovered, it should be corrected. This may involve revising the code over and over. One of the most important insights since the early days of programming is that we all make mistakes, regardless of how well we've studied a task and how carefully we've planned. It's good to revise code.

Another trivial fact, sometimes overlooked, is that understanding the task is crucial. For example, discussions between clients and programmers may involve entirely different meanings for a task. Regardless of how neatly the programmer solves the task according to their definition, the client may be dissatisfied. Returning to the pancake example, a client with an American background probably doesn't mean the same thing with the word "pancake" as a Swedish programmer who might translate it to "pannkaka." Solving this challenge is far beyond the scope of this course, but it's worth noting that even if you program for yourself, you may discover more aspects to a task than you initially understood.

5.1.3 Work with the assignment!

Here are some considerations, which may seem trivial but are valuable to go through, especially when working together with others:

1. What is the input to the calculation? For example, if you want to count the number of words in a text, how is the text represented? Is it given as a single long string? Or perhaps as a list of strings, where each string represents a sentence? It's also good to consider edge cases. For the word-count example, is it possible for the text to be empty, i.e., have a length of zero, or is there always at least one word? When going back to the pancake algorithm, do we always know that we have enough ingredients? It might be good to adjust the recipe based on the actual number of eggs available.
2. How should the result look? In general, when writing functions to solve tasks the function should return a value (we will discuss this further in the chapter on program structure). The value can consist of a lot of information. For instance, if you write a function that counts the number of words in a text, returning an integer may be sufficient, but it might be more useful to return a dictionary associating words with an integer describing how many times they appear in the text.

What we are discussing here is the start and endpoint of a calculation. But how do you figure out what comes in between? That's where the algorithm itself is, and it's not always obvious to come up with it. Some algorithmic tasks are simply very challenging to understand and find solutions for. In this course, the tasks are relatively simple, in the sense that high school knowledge in mathematics is sufficient preparation. Regardless of difficulty, there are ways to work your way to an algorithm.

It is possible to work towards algorithms, and they don't have to be complicated to solve problems that are difficult to describe. Here are some simple tips to make progress:

- Create small examples. These examples can be useful for experimenting when you want to figure out how an algorithm should look, and they can be helpful when testing your code.
- How do you actually solve a task in your head? Some things, like addition, we are so good at that we have stopped thinking about the steps. A careful review of the steps you are comfortable with can give you the solution.
- How would you solve the task with pen and paper? Experiment with it!

Working on the task is important to understand the problem, and it can be counterproductive to start writing code right away. If you don't understand the problem, your code is likely to be wrong. On the other hand, coding can be a way to discover that you haven't understood the problem.

Finally, make sure to understand the code you write. You shouldn't wish your way to working code. If you think, "this function should give me the answer" or "I believe this works," you should slow down. Words like "should" and "believe" are warning signs. You should know that the code works or at least

have good reasons to believe that it does. If it feels shaky, simply test it. Verify that functions do what you think they do. Reason with yourself about what should happen in your code. Besides determining what works and what doesn't, it's a good way to learn programming.

5.2 Data Representation

Often, one doesn't need to ponder how data is stored in a computer. When working with integers, for example, it's sufficient to have an abstract image that numbers are stored as zeros and ones. For instance, writing `x = 7` allows us to easily understand that the integer 7 "exists" in `x`. The rules for how zeros and ones form integers are called integer representation and are standardized today (though Python deviates, as explained in later chapters).

Built-in types in Python (and other programming languages) are easy to use without concerning oneself with their representation. However, for more problem-specific data (e.g., people's addresses) and abstract ideas (like polynomials and other mathematical concepts), the programmer must think and find ways to use the language's data structures and other features to find a suitable data representation. In this chapter, we discuss some examples of how to work with data representation.

5.2.1 Learning Objectives

- Representation of integers, floating-point numbers, strings, and boolean values
- Other data
 - How to store coordinates?
 - How can polynomials be stored?
 - Storage of date and time

5.2.2 Primitive Data Types

Many data are represented straightforwardly in programming languages because there are built-in types for them. The most fundamental types are usually called primitive data types, and in Python, these include `bool`, `int`, `float`, `complex`, and `str`. Different languages have different primitive data types; in the C language, for instance, `char` is a data type representing a character in exactly one byte, and strings are (approximately) lists of `char` values. Python does the opposite, as even a single character is stored in a `str`.

Lists and dictionaries are called composite data types, as they, in some sense, consist of primitive data types.

5.2.3 Data Structures

A data structure is a construction that can gather several primitive data types into a unit. You have already encountered lists, tuples, and dictionaries designed to collect various amounts of data so that parts are easily accessible. These data structures are generic, and their purpose is to make it easy to work with data of variable size. Still, they can also be seen as a convenient technique for indexing or naming parts of data. There are other data structures in Python, like the concepts of `classes` and `objects` providing a way to create custom data structures. Classes are outside the scope of the course.

5.2.4 Represent Abstractions

Most people may not find it strange to store concepts like *name*, *address*, and *text* as strings. However, representing more abstract concepts can be challenging.

As a first example of abstractions, consider points in a plane. We might want to store positions on a map, points in a figure, etc., and need a way to work with the coordinates of these positions. One approach is using two variables per point, like *x* and *y*, or two lists *x* and *y* for multiple points. However, this method separates the two coordinates, which can be impractical and error-prone.

An alternative is storing coordinates as “pairs”, or rather lists with two elements. For instance, defining `origin = [0, 0]` or `points = [[0,0], [1,0], [2,1]]` groups values naturally, making operations like `origin = points[0]` straightforward.

Another option is using a dictionary for points, initializing `origin = {'x': 0, 'y': 0}`. This way, elements can be named, improving readability. For example, `p['x'] = p['x'] + delta` for adjusting the *x*-coordinate is more understandable than `p[0] = p[0] + delta` using lists.

This discussion revolves around how points in the plane are represented. It involves taking a highly abstract concept (points), considering the necessary data (*x* and *y*), and determining how to use simple data structures for actual storage.

5.2.5 Example: Personal Identification Numbers (Personnummer)

In many public IT systems, dealing with personal identification numbers (personnummer) is common. How can we best represent them? Since personnummer can be written with 10 or 12 digits, considering them as integers is an option. For instance, the fictitious personnummer 890201-3286 would become 8902013286. However, this can lead to issues with the range of a 32-bit integer in most programming languages. An alternative is splitting the number into two parts, like `[890201, 3286]`. This overcomes the range problem, but it still has complications for people born in the 2000s. While we will certainly be able to compare personal identification numbers without any issues, there will be misunderstandings during printouts since humans expect personal identification numbers to be either 10 or 12 digits long.

It is now time to take a step back and consider what we may want to do with personal identification numbers. In most cases, we simply want to compare them (is this the correct person?), and performing arithmetic operations on them is rarely essential. Therefore, there is no advantage in storing personal identification numbers as integers. The exception is when verifying that a personal identification number is correctly inputted, but even then, it is the individual digits used for verification. We might want to ensure that the date part of the personal identification number is a valid date. Many programming languages provide excellent support for storing and working with dates, often having specific types for this purpose. In Chapter 6, we introduce *modules*, and in Python, the `datetime` module is useful for working with dates. However, those familiar with personal identification numbers will soon recall that the Swedish Tax Agency (“Skatteverket”) uses “samordningsnummer” for individuals not registered in Sweden. Samordningsnummer should be compatible with personal identification numbers but still distinguishable. Therefore, the date of birth remains the basis for samordningsnummer, but 60 is added to the birthdate. If we convert our example personal identification number 890201-3286 to a samordningsnummer, it would be written as 890261-3286. Representing such a birthdate is not possible with a standard date type.

Perhaps the simplest approach is to represent personal identification numbers as strings?

5.2.6 Example: Polynomial

When working with polynomials (which may be needed not only by mathematicians), a representation for them is necessary. It is not practical to store $4x^2 - 3x + 7$ as a string, as it complicates calculations. A common way to work with polynomials is to store the coefficients as numbers in a list, with the coefficient for the term of degree i at index i in the list. It may seem counterintuitive, but the advantage is that the length of the list precisely corresponds to the degree of the polynomial. Additionally, there are algorithmic advantages.

Here are some examples to illustrate:

Polynom	Python-representation
$3x^2 - 2x + 1$	[1, -2, 3]
x^4	[0, 0, 0, 0, 1]
$4x^2 + 5x^3$	[0, 0, 4, 5]
$5 + 4x + 3x^2 + 2x^3 + x^4$	[5, 4, 3, 2, 1]

A mathematician might be concerned about how we represent the zero polynomial. Is it like [] or [0]? It's a special case that may need handling, especially in the calculation of a polynomial's degree, but that's a detail we won't delve into here. With the representation of polynomials as coefficient lists, we have the ability to perform various operations. Adding two polynomials is simply done by adding the elements in two coefficient lists in this representation, with slight consideration that the polynomials may have different degrees.

Let's look at the differentiation of polynomials as an example. We want to differentiate symbolically, without calculating the derivative at a specific point, so the result should be a new polynomial. How did we learn it in school? Well, we learned that the constant term disappears, a term ax^b is differentiated to abx^{b-1} , and each term can be differentiated individually. We can iterate through a coefficient list, keep track of the degree of the term represented by the coefficient, and calculate new coefficients like ab . A simple algorithmic idea for symbolically differentiating a polynomial p given as a coefficient list:

- Skip the coefficient at index 0.
- Keep track of the current degree.
- Add the new coefficients to a new list.
- Iterate over the coefficients from index 1 and calculate the new coefficients.

We can formulate Python code almost directly from this algorithmic idea:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial[1:]:
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Om man testar funktionen så kan det se ut så här:

```
derivative([1, -2, 3])
```

`[-2, 6]`

This means that we have transitioned from an abstract polynomial to a concrete symbolic representation, mechanically derived to a new coefficient list, representing another abstract polynomial:

$$3x^2 - 2x + 1 \longrightarrow [1, -2, 3] \longrightarrow [-2, 6] \longrightarrow 6x - 2$$

We can conclude that with the right representation, we can perform advanced calculations on abstract concepts.

5.3 Debugging

When programming, one encounters mistakes, misunderstandings, and misjudgments. This creates errors in the code, sometimes referred to as “bugs”. The term has been used to describe errors in technical systems even before computers were built. Bugs can cause programs to crash, abruptly terminating the program because it cannot continue, or simply result in incorrect output that may go unnoticed. It is essential to appreciate error messages; it’s far worse to be unaware of errors.

Programming errors can be categorized as syntactical mistakes and runtime errors (logical mistakes). Syntactical mistakes (e.g., writing `l = [2 4]` instead of `l = [2, 4]`) are caught by the compiler and are usually easy to understand once you’ve learned the programming language. Runtime errors can be more challenging, as one often discovers symptoms or consequences of mistakes or problems. The error message received may not always reveal the actual (logical) error but serves as a symptom. For example, if a program crashes because a lookup table lacks the key used for a lookup, the lookup itself may not be the error. Instead, it could be an incorrect key or the key not being inserted. Addressing symptoms without solving the underlying problem is seldom the right approach in programming.

5.3.1 Examples of Errors

Errors often arise from misunderstandings, whether it be a detail in an algorithm, a mistaken idea about input data, or a misunderstanding of the programming language being used. Here are some examples of recurring errors to illustrate what one needs to be attentive to.

5.3.1.1 Example: Sorting Issues

Suppose you need to sort a list of numbers and are experimenting with the following code snippet:

```
lst = [10, 3, 4]
lst = lst.sort()
for elem in lst:
    print(elem)
```

If you run this small program, you will get an error message in the for-loop: `TypeError: 'NoneType' object is not iterable`. Why is that? It’s evident that there are elements in the list, right? The error occurs due to a misunderstanding of how `sort` works. The expression `lst.sort()` returns `None` but changes the order of elements in `lst`, making line two in the code unfortunate because `lst` is then assigned `None`.

Note that the error is made on line two but detected on line three. This is common: when we observe errors, they are often symptoms of mistakes made earlier. One way to correct the above error is to switch to the `sorted` function.

```
lst = [10, 3, 4]
lst = sorted(lst)
for elem in lst:
    print(elem)
```

This works since sorted returns a sorted copy of the list.

5.3.1.2 Example: Poor Return Value

One of our rules of thumb is always to return a result from a function. There should be good reasons not to return anything. The reason is to avoid a common beginner mistake, which even experienced programmers can make.

Let's assume we want a small function that extracts the three largest values from a list. We experiment and find that it is easily solved with two lines of code:

```
lst = [10, 3, 4, 5, 1, 2]
lst = sorted(lst)
print(lst[-3:])
```

```
[4, 5, 10]
```

Good, let's make it a function:

```
def largest_three(lst):
    lst = sorted(lst)
    print(lst[-3:])
```

We test the function, as you should, but an error appears

```
a, b, c = largest_three([10, 3, 4, 5, 1, 2])
```

```
[4, 5, 10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot unpack non-iterable NoneType object
```

The result is printed, but the assignment of the return value does not work because there is no return statement, resulting in the return value being None. One should always return the relevant values!

```
def largest_three(lst):
    lst = sorted(lst)
    return lst[-3:]
```

5.3.1.3 Example: Iterators are not Lists

Inspired by the solution above, one might consider a similar task solved in a similar way. Let's assume we want to print a list in reverse order and display the maximum value.

```
lst = [10, 3, 4]
lst = reversed(lst)
for elem in lst:
    print(elem)
print('Largest:', max(lst))
```

Again, an error occurs on the last line: `ValueError: max() arg is an empty sequence`. Why is that? There is no error if we use `sorted` instead of `reversed`. However, these two functions have different types of return values. `sorted` returns a `list`, which can be used repeatedly without any problem. On the other hand, `reversed` returns an iterator, which is used only once. When the `for` loop is done, the iterator is “consumed” and `max` has nothing to work with.

This is a common mistake. Iterators are widely used in Python for their efficiency and memory-saving properties, but one must be attentive to the fact that an iterator sometimes needs to be converted to a list.

How do you correct this mistake?

5.3.2 Understanding the Error Message

A crucial step in becoming proficient at debugging is carefully reading error messages. If a program terminates with an error, the error message usually indicates clearly why it is terminating. Again, this is often just a symptom of the actual error, but symptoms are crucial to getting to the source of the error.

Python error messages always follow the same structure:

- First, the location of the error is shown, indicated by a *stack trace* that specifies the involved function calls and their locations in files and on which lines they are called.
- Finally, the file name, line number, and a simple description of the error is shown.

Consider this peculiar code, written solely to showcase an error message.

```
def f1(x):  
    return f2(x)  
  
def f2(x):  
    return f3(x)  
  
def f3(x):  
    return 17 / x  
  
f1(0)
```

If you execute the program, it calls `f1(0)`, which then calls `f2(0)`, and subsequently calls `f3(0)`, leading to a division by zero. This results in the following error message (at least in Python 3.9):

```
Traceback (most recent call last):  
  File "/Users/ksahlin/code.py", line 10, in <module>  
    f1(0)  
  File "/Users/ksahlin/code.py", line 2, in f1  
    return f2(x)  
  File "/Users/ksahlin/code.py", line 5, in f2  
    return f3(x)  
  File "/Users/ksahlin/code.py", line 8, in f3  
    return 17 / x  
ZeroDivisionError: division by zero
```

Quite a bit of text for a simple error, but it’s illustrative. Pay special attention to the fact that it indicates on which line the error occurs, here on line 8, and that the symptom is specifically division by zero. Editors

helpfully display the problematic file and line. And thanks to “Traceback,” which gives us our stack trace, you can see how functions have called each other, providing additional tips for troubleshooting.

Some error messages can be challenging to read, and for a beginner, certain terminology may be too technical. However, even if you don’t understand everything, you can still find valuable clues.

5.3.3 Troubleshooting with Trace Prints

Trace prints are a proven and widely used technique for error debugging. The idea is simple: insert print-statements that reveal variable values, showing both how variable values change and how the program flow progresses. It can be viewed as a meticulous way to identify symptoms that can disclose the nature of the error.

As an example, let’s look at the polynomial symbolic derivation example from the Data Representation section (section 4.5), but with an injected error:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:    # Error!
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Assume that we have observed that something goes wrong without managing to figure out what the cause is. A simple trace print shows the relevant variable values in each iteration:

```
def derivative(polynomial):
    result = []
    degree = 1
    print('degree term new_coefficient')    # Practical header
    for term in polynomial:    # Error!
        new_coefficient = term * degree
        print(degree, term, new_coefficient) # Trace Prints
        result.append(new_coefficient)
        degree += 1
    return result
```

This trace print would hopefully quickly reveal that we have mistakenly included the constant term in the derivation.

When working with trace prints, it’s easy to feel the need to spice it up with explanatory information. Here above, for example, it was good to include a headline. The need is so common that Python has a special feature for trace prints somewhat hidden in f-strings: if you write `f'{x}'`, `{x}` is replaced by the value of the variable `x`, but if you write `f'{x=}'`, `x=` is set instead, including the value of the variable. So, the example above can be better written like this:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:    # Error!
        new_coefficient = term * degree
```

```

    print(f'{degree=}, {term=}, {new_coefficient=}') # Trace Prints
    result.append(new_coefficient)
    degree += 1
return result

```

Try it! If we run this with the polynomial $1 + x + x^2$ it will look like this:

```
derivative([1,1,1])
```

```

degree=1 term=1, new_coefficient=1
degree=2 term=1, new_coefficient=2
degree=3 term=1, new_coefficient=3

```

Here one should notice that there is a term with degree 3, which is indeed wrong since we have a quadratic polynomial as input.

Trace prints are convenient and practical to use but are often considered bad practice. The disadvantages of trace prints are that it is laborious and sometimes unnecessary to insert them; one usually has to remove them as well. However, they are still widely used by experienced programmers. In scripting languages like Python, the cost of trace prints is relatively low since one tends to work quite interactively with the code, more than in some other languages, and it is easy to test the code.

5.3.3.1 Logging

If one finds that trace prints are useful, they might enthusiastically start planning for specific trace print functions that are toggleable, where the trace prints can be written to a separate file. In that case, logging has been initiated, and there is no reason to write custom code for it. It is such a common need that there is a standard module, named logging, for this purpose. The purpose of the module is to make it easy to log program status, not just for error tracking but more generally to keep track of what a system is doing. Many computer systems work over a long time with essential system services, and logs become a crucial tool to monitor what has been done in a system.

Here is an example of how the derivative code can be adjusted to log status to the file `derivative.log`:

```

import logging
logging.basicConfig(filename='derivative.log', level=logging.DEBUG)
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:
        new_coefficient = term * degree
        logging.debug(f'{degree=}, {term=}, {new_coefficient=}') # Spårutskrift
        result.append(new_coefficient)
        degree += 1
    return result

```

The two initial lines initialize logging, indicating the file where log messages will be stored and the logging level to be used. Without `level=logging.DEBUG`, debug messages won't have any effect. However, prints made with `logging.warning` or `logging.error`, two other print methods with higher priority, will still be output.

If one evaluates the code above and then calls `derivative([1, 1, 1])`, these prints will be found in the log file:

```
DEBUG:root:degree=1, term=1, new_coefficient=1
DEBUG:root:degree=2, term=1, new_coefficient=2
DEBUG:root:degree=3, term=1, new_coefficient=3
```

In the left margin, the type of log message (“DEBUG”) is indicated, and the term “root” denotes the source of the message, which, in our example, doesn’t have any function. The module supports more advanced usage than demonstrated here.

5.3.4 Debugging with a Debugger

The programmer’s perhaps best tool is the debugger. It’s a program used to run your program in a controlled way, allowing you to step through it, line by line, and inspect variable values without trace prints.

In Python-specific editors like Spyder and PyCharm, the debugger is built-in. Other editors, such as VS Code, have add-ons that provide debugger support. Some tools always have debugger controls available, while in others, you must instruct the editor explicitly to “start debugging.” Regardless of the editor and debugger used, there are recurring and useful features for most programming languages. Let’s look at some fundamental features.

5.3.4.1 Breakpoints

A breakpoint is often marked with a red symbol in the left margin of the code, signifying “stop before this line is executed.” You can start your program with the debugger, and when the execution reaches the breakpoint, it pauses. The execution is not completely halted but can be resumed when you are ready.

During the pause, you can inspect the values of various variables, providing clues about the error. Working with breakpoints and variable inspection is somewhat similar to using trace prints, but the advantage is that you don’t need to remove trace prints when you are done. If your program has many variables with somewhat complicated data structures, breakpoints are often easier to work with. An additional benefit is that you can investigate data structures without having to pre-select what should be displayed.

5.3.4.2 Stepping through the program

Breakpoints provide snapshots of the program, but they may not always be sufficient to explain what is going wrong. Stepping through the program line by line, inspecting variables when necessary, or simply understanding the program’s flow can be practical.

There are two ways to step through code, and it’s advisable to use both. In the simple step, often called “step” in a debugger and available via a button, the current line is always executed. If the line contains a function call, you follow the code into the function, sometimes delving into Python’s internals. This can be confusing and time-consuming. It’s rarely necessary to examine Python’s own implementation to understand why your program is failing. In such cases, it’s good to use the other step method, often called “next” or “step over.” With “next,” you instruct the debugger to execute code lines and continue to the next line, skipping the details of the function that may be called.

There is usually an option to “continue to the last line of the function”, which is a faster but still controlled way to move forward.

5.3.5 Strategies for Debugging

Even if you master debugging techniques, finding the real mistake can be challenging or slow. It's worth considering how to reveal the underlying error.

In the section on Defensive Programming, tips are given on using `assert` to insert conditions into your program. Conditions should describe statements that should be true where the `assert` statements are located. If the condition is made false by a bug, the program is aborted.

You can also read about unit testing, a systematic and automated way to test that the program's functions are correct. If all functions in a program have unit tests, it helps to find mistakes early.

With or without unit testing, it will always be important to use effective strategies to find errors. We will now go through some strategies.

5.3.5.1 Delimiting Errors with “checkpoints”

If a program crashes on line 917 after executing several different functions, the mistake may just be on that line or the line before. However, one should not assume that the error is even close to line 917. Errors can propagate in multiple directions without notice (logical errors). It becomes important to lift your gaze and look further back in the program. In that case, examining first line 917, then line 916, and line 915, and so on backward, is slow. It is more efficient to, if the error is not close to line 917, try to delimit. Are the inputs to the last-called function correct? How were these inputs created?

When the error is really confusing and hard to find, you may need to create “checkpoints” in your program to verify that the calculations are correct up to them. This way, you can delimit and isolate the source of the error.

When you have a working program that you are expanding or modifying, the new code is, of course, the main suspect and should be reviewed first.

5.3.5.2 Rewrite the code

Debugging involves addressing errors in code, and sometimes, the mistakes are right in front of us but remain unseen due to incorrect assumptions or confusion. Confusion and temporary blindness to certain mistakes are common, considering the complexity of holding multiple concepts in one's mind while programming.

One common type of mistake, particularly in scripting languages (unlike languages like Java where variable declarations are mandatory), is spelling errors. In simplified terms, this might look like the following:

```
identifier = 7           # A first assignment
idenfifier = identifier * 2 # Updating the assignment
print(identifier)
```

Why is the value 7 printed? Can you spot the mistake?

If you can narrow down an error to a function but still struggle to identify it, a strategy is to rewrite the function. The process of rewriting a function, perhaps with different identifiers or a slightly different algorithm, can cure temporary blindness to simple mistakes like typos or reveal underlying assumptions that were initially overlooked.

5.3.5.3 Review the input data

Some issues arise because input data contains peculiarities that may be challenging to notice. While modern standards have largely addressed this, problems can still occur, such as when data is created on a Mac but analyzed on Windows.

In most cases, relying on UTF-8 character encoding for text files is safe, compatible with older encodings like ASCII. However, in older Python versions (2.7 and earlier), automatic UTF-8 handling may result in odd characters, particularly with Swedish characters.

Assumptions about the input format are common pitfalls. Details like whether spaces are coded with a “space” or a TAB character can be easily overlooked when inspecting data in a text editor. Writing code to handle both spaces and TAB characters can be beneficial.

Moving files between systems may reveal differences in how line breaks are handled. While Mac and Linux use `\n` in Python, PCs use `\n\r`, reflecting historical connections to early computer printers.

To avoid such intricacies, relying on established code for data reading is advisable, as discussed in a later chapter.

5.3.6 Choose Your Test Data Carefully

When debugging, selecting proper test data is crucial:

- Work with small test data to expedite debugging, even if the goal is to analyze large files.
- Errors often arise with uncommon inputs, so testing constructed special cases is beneficial.
- If reading data from a file, ensure your code can handle an empty file without crashing. Functions working with strings should handle the empty string `''` seamlessly.
- If implementing a numerical function $f(x)$, it should function correctly at $x = 0$ if defined there.

Consider that it is likely that you will have to test your program many times, maybe even in the future. It is therefore wise to construct and save test data for future needs.

5.4 Exercises

In the following exercises you will be given a series of buggy code examples, your task will be to identify these bugs by reading the code line by line.

While reading, you will have to:

- Recognize the bug,
- Identify the cause of the bug,
- Propose a fix

1. The buggy `all_zero` function.

```
def all_zero(p_list):
    """ Check if all elements are zero.
    :param p_list: list of integers
    :return: boolean
    """
    for x in p_list:
        if x != 0:
            return "False"
```

```
else:
    return "True"
```

- What are the bugs in this code?
- What tests can we write to find the bugs?
- How can we fix the bugs?

2. The buggy add_hyphens function.

```
def add_hyphens(text):
    """ Adds a hyphen between letters
    :param text: string
    :return: string
    """
    new_text = " "

    for idx in range(text):
        if idx == 0:
            # add first letter
            new_text = text[i] + "-"
        elif idx == len(text):
            # add last letter
            new_text = text[idx]

    new_text = text[idx] + "-"

    return new_text
```

- What are the bugs in this code?
- What tests can we write to find the bugs?
- How can we fix the bugs?

Chapter 6

Error handling/exceptions

Things go wrong sometimes, in programming as in life, and we have to handle it. When it comes to programming, it is typically the status of a function we are interested in: was a computation successful, or was there an error? How do we continue after an error?

Until now, we have defined functions that return a result (value), while instructive messages are perhaps written to the user with `print`. Alternatively, a program might simply crash or get stuck in an endless loop.

In this chapter we will see how code, upon an error, can call (or “raise”) an exception. These can be handled using **error handling**, which in Python is expressed using the **try** and **except** keywords.

6.1 Error handling and exceptions

A naive way of handling errors in a program is to return some kind of error code in the form of a number or string. We will first take a look at how that can play out, but quickly move on to see how errors can be handled more effectively with the help of exceptions.

6.1.1 Error handling via returned error codes

Using special return values for error handling leads to complex programs that tend to be hard to read. Many function calls must be followed by **if**-statements testing whether an error has occurred that needs to be dealt with. If an error has occurred, it can be tricky to return to the right place in the program from which to continue. Experience shows that programmers end up avoiding the increased complexity by ignoring returned error codes, which leads to unstable programs that can crash. This type of error handling is often found in C, Fortran, and other older programming languages.

An example of this old-fashioned technique follows below, where we call a function that modifies a list. Any return value other than 0 signals that an error has occurred.

```
def divide_list(l,x):  
    if type(x) != int or type(x) != float:  
        return 11  
    elif x == 0:  
        return -1
```

```

else:
    # check that elements in list are integers
    if not all([ type(item) == int for item in l]):
        return 12
    for i in range(len(l)):
        l[i] = l[i]/x # we are modifying the list items here
    return 0

return_code = divide_list(l,x)
if return_code == -1:
    # do something
elif return_code == 11:
    # do something else...
elif return_code == 12:
    # do something else...
elif return_code == 0:
    # good!

```

This code is fairly messy and unnecessarily complicated. Many modern languages therefore support specific tools to handle errors via so-called *exceptions*.

6.1.2 Exceptions

Python makes use of the reserved keywords **try** and **except** to pair up “unsafe” code that might raise an exception with code to be run in that case.

```

try:
    # unsafe code
    ...
except:
    # action
    ...

# Rest of the code
...

```

The keyword **try** opens a block of code regarded as unsafe (*# unsafe code* above), while **except** encloses the measures to be taken if an error arises. Just as with function definitions, conditionals, and loops, the indentation is essential. The block belonging to **except** (*#action*) is executed only if an exception is raised in the unsafe passage within the try-block: if no error has occurred, the execution continues below without running the code in the **except**-block. If an error occurs, then the code in the **except**-block is run. It might end the program or handle it in another way—perhaps even ignore it, though one should not do so carelessly.

6.1.2.1 Example: reading an integer from the user

Consider the following code snippet:

```

while True:
    answer = input("Write a number (write 0 to quit): ")
    x = int(answer)

```

```

if x == 0:
    print("Bye bye!")
    break
else:
    print("The square of the number is: " + str(x ** 2))

```

What happens if the user enters something other than a number?

Write a number (write 0 to quit): hej

ValueError: invalid literal for int() with base 10: 'hej'

As we've seen earlier, this results from the fact that the value of input always has the type `str` and that it is not always possible to explicitly coerce from `str` to `int`. Here, **ValueError** is an exception: there is something wrong with the value of the argument ('hej') to `int`. Using **try/except**, we can handle the error in some appropriate way:

```

while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except ValueError:
        print("You must write a number!")

```

In the code above, any exception of the kind **ValueError** raised in the **try**-block will be caught. If some other kind of error is raised, the program will terminate and display the error as before. We can also catch *all* errors by using an unqualified **except**:

```

while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except:
        print("You must write a number!")

```

In order that the code be predictable and clear, however, one should write as little code within the **try**-block as possible. For example, we can move the `if`-statement in the code above out of the **try**-block like so, making it clearer where we expect an exception might be raised:

```

while True:
    answer = input("Write a number (write 0 to quit): ")
    try:

```

```

    x = int(answer)
except ValueError:
    print("You must write a number!")
    continue

if x == 0:
    print("Bye bye!")
    break
else:
    print("The square of the number is: " + str(x ** 2))

```

6.1.2.2 Example: calculating the lengths of lines in a file

Suppose we want to calculate the length of each line in a file:

```

def len_file(filename):
    with open(filename, 'r') as h:
        for s in h:
            print(len(s))

len_file("foo.txt")

```

If the file `foo.txt` does not exist, we receive the following error:

```
FileNotFoundError: [Errno 2] No such file or directory: 'foo.txt'
```

It is therefore better to write:

```

def len_file(filename):
    try:
        with open(filename, 'r') as h:
            for s in h:
                print(len(s))
    except FileNotFoundError:
        print("Could not open file " + filename + " for reading.")

len_file("foo.txt")

```

6.1.2.3 Different actions for different exceptions

There are many things that can go wrong. Depending on which exception is raised, it could be appropriate to take different actions. In the example below, we respond differently depending on whether something goes wrong with file reading, division by zero, or coercing a string read from a file to an integer. Note that the **try-except** need not be placed exactly where the error occurs.

```

def divide_by_elems(filename, x):
    quotients = []
    with open(filename, 'r') as h:
        for n in h:
            frac = x / int(n)
            quotients.append(frac)

```

```

    return quotients

try:
    data = divide_by_elems('numbers.txt', 2)
except IOError:
    print("divide_by_elems: A file-related problem occurred.")
except ZeroDivisionError:
    print("divide_by_elems: Division by zero.")
except ValueError:
    print("divide_by_elems: Character could not be converted to int.")

print(data)

```

For example, if the code above were run with a file `numbers.txt` containing the following:

```

1
2
0
4
3
0

```

then an exception would be generated and caught. Which? Can you make changes so that each of the other exceptions is raised?

6.1.2.4 Exceptions for control flow

We can also use exceptions for control flow. Suppose we want to calculate the quotients of the elements at each index of two lists (and ignore all elements at the end of the longer list), but not crash if there is a division by zero. To do so, we can write the following:

```

def compute_ratios(xs,ys):
    ratios = []
    for i in range(min(len(xs),len(ys))):
        try:
            ratios.append(xs[i]/ys[i])
        except ZeroDivisionError:
            ratios.append(float('NaN')) # NaN = Not a Number
    return ratios

print(compute_ratios([2,3,4],[2,0,5]))
print(compute_ratios([2,3,4,0],[2,0,5]))

```

In this case, we could have equally well used an `if`-statement and tested if `ys[i] == 0`. We'll say more on this later.

6.1.3 General advice on exceptions

It is considered idiomatic Python to make use of `try-except`. It is not unusual when reading Python code to see *EAFP*, which stands for *it's Easier to Ask Forgiveness than Permission*. This is contrasted with

Look Before You Leap (LBYL). This does not mean that you should *overuse* **try-except**: raised exceptions can sometimes give the clearest information. Not every exception in code is meant to be handled. The best time to use **try-except** is when you have an unsafe passage and a clear idea of what should be done in case of error.

So what is an unsafe passage in a program? Some typical cases are:

- All file handling
- Communication with the user
- Code that mentions insecurity in its documentation (“throws exception if...”)
- Code you discover is unsafe (for example, implements an unstable algorithm)

There are several common actions to take after catching an exception:

- **Print an error message and exit:** used for example if data is missing or the error is so serious that there is no reasonable way to continue.
- **Print a warning, ignore the error, and continue:** if the program has received strange input data, but can ignore it and continue computation anyway.
- **Handle the error and continue:** works well for some foreseeable errors. For example, if you get bad input from a user, you can simply ask for new input.
- **Create a new exception:** This is appropriate if you have code where several errors can occur, but these can be organized into more informative groups. If an algorithm is unstable, `ArithmeticError` might be a better error than `ZeroDivisionError`: perhaps the user of the algorithm knows about the algorithm’s general instability, but won’t understand what it has to do with division by zero.

There are also actions that you should avoid:

- **Don’t ignore the error!** The worst thing you can put in an **except** block is **pass**. It then becomes very difficult for the user understand what went wrong and where it happened. Python would have helpfully raised an exception, but the programmer decided to suppress it. The error may cause problems down the line, and it may be difficult to trace them back to the source.
- For **serious** errors, it is *not enough* to print an error message and continue. It isn’t useful to let a program continue if the error is so serious that you know it will cause another error later. Some errors should simply end the program.

6.1.4 Managing your own uncertainty

It is not always necessarily a bad thing for code to raise an error. It is difficult, if not completely impossible, to for example

- Predict the format of *all* possible inputs.
- Ensure that all users (including yourself) call functions correctly.

Python includes many specific exceptions (see the [documentation](#) for a detailed accounting). Some of the common ones you will encounter are:

- `TypeError` (e.g. `'hej' + 32`)
- `ZeroDivisionError` (e.g. `3/0`)
- `KeyError` (e.g. `d['hej']` when ‘hej’ is not a key in d)
- `IndexError` (e.g. `l[4]` when `l = ['a', 99, 2, 'b']`)
- `SyntaxError` (e.g. `l[4` or `def f()` without colon)
- `ValueError` (e.g. `int('hello12')` or `import math; math.sqrt(-10)`)
- `FileNotFoundError`
- `IOError` (when reading/writing to file)

In Python, an exception is not just a text string or error code; exceptions are a class of their own. (We will talk about classes in more detail in a later chapter.) This makes it possible to easily create your own exceptions based on the existing ones.

You can raise an exception with the reserved keyword **raise**. (In Java or C++, the analogous operator is **throw**.) For example, if we want to get the first element of a list but throw an error if the list is empty, we can write:

```
def head(xs):
    if not xs:
        raise Exception("head: input list is empty")
    else:
        return xs[0]

try:
    print(head([]))
except Exception as e:
    print(str(e))

print(head([]))
```

```
head: input list is empty
# specific information about where the exception was caught
Exception: head: input list is empty
```

Note that the code continues after the *first* time `head` is called in the **try-except** block: only the *string* with the error message is printed. We can “save” the exception in a variable that we can later use (for information, for example, or if we want to re-raise the exception after say saving something to a file) by writing **except Exception as e**; here `e` is an identifier to which we assign the caught exception. The string with the error message can then be obtained by writing `str(e)`. The *second* time `head` is called above, the exception occurring in the function is raised without being caught. The same information is printed again, but now it is the raised **Exception** that delivers the message, rather than the code in the **except**-block, and the program exits immediately after.

The keyword **raise** is useful for writing more detailed error messages, or when an error occurs that isn’t covered by Python’s existing error handling. An example of such an error is when an algorithm that we have written must maintain a value greater than 0.01 in a parameter `h` if the algorithm is to work. (Think for example of a step length on the x-axis for finding zeros of a polynomial.) We can write something like

```
def my_algorithm(x, h):
    while True:
        if h < 0.01:
            raise ArithmeticError('Stepsize h should not be smaller than 0.01.\
The value of x was:', h)

        # check polynomial root
        ...

        # update h
        ...
```

6.1.5 How you deal with exceptions depends on what you're writing

There are different scenarios where we might want to use exceptions differently.

- **Application programming:** If you write a program that uses code from different modules, both your own and others', then you will encounter functions and methods that generate exceptions. It then becomes important to use **try-except** to deal with exceptions when they arise. The program should not exit abruptly because of a problem that could be foreseen and perhaps easily dealt with. You also want to avoid forcing a user to understand a Python error message.

Some examples:

- If an exception occurs because of bad input, then you should tell the user in a clear way. Letting through an error message from an exception is not user-friendly, because the cause of the problem is often not at all clear from the exception that was raised.
- If an exception occurs because of, say, numeric uncertainty or uncertainty in the environment (for example, because a file with some name already exists), then you should probably deal with this somehow rather than throwing an exception in the user's face.
- **Support programming:** It is very common to write separate code as support for the program you actually want to write. If your goal is to write your own version of Matlab, then you'll need many functions that perform calculations; if you want to analyse data stored in a complicated format, it might be good to have a module that just handles the data so that you can later focus on the calculations. These support routines can run into problems that must be reported to the user (that is, the main program), and raising an exception with **raise** is an appropriate way to do so.

Python will report *that* an error has occurred. As the programmer, you can report *why* an error has occurred.

6.1.6 Common missteps

6.1.6.1 Use if-statements instead of exceptions for control flow

It can be tempting to use **except** for control flow, but often it is better to instead use an **if**-statement. For example, consider this code:

```
def elem_access(xs,i):
    try:
        if i >= len(xs):
            raise IndexError
    except IndexError:
        print("Index is out of range.")
    return xs[i]
```

There is clearly no point in first raising an exception and then immediately catching it. Another mistake in this example is that if there is an error, the program will *continue* after printing an error message, here trying to access `xs[i]` even though `i` is too large. If you catch an exception, then you should handle the problem in some way. It is often appropriate to end the program by printing an error message and calling `quit()`.

6.1.6.2 Overuse

Don't use exceptions to patch up code that isn't behaving as you expected.

In the example below, we create a new dictionary with keys `e` taken from a list selected by calling a function that uses the information in another dictionary `data`. Clearly the programmer has noticed that not all values in `selected` exist in `data` and therefore tried to compensate by catching the `KeyError`'s that arise:

```
data = {1 : "Make bed",
        2 : "Shower",
        4 : "Eat breakfast"}
selected = [1,3,2]

d = {}
for e in selected:
    try:
        d[e] = perform_task(data[e])
    except KeyError:
        print()      # Bad line of code!
```

Instead, one should try to eliminate the underlying cause of the error.

6.2 Exercises

1. What happens if we call the function `divide_by_elements` (the function given in section 6.1.2.3) with a string as the argument to `x`? If the string can be converted to an integer, let us say the function should continue without crashing; otherwise it should return `None`. Achieve this behavior with the help of `try` and `except`.
2. Rewrite the function `divide_by_elems` so that it does not break after the first error, but continues to divide the number `x` with lines that come after. For example, for the file `numbers.txt` as above, the function should return the list

```
[2.0, 1.0, 0.5, 0.6666666666666666]
```

Tip: Replace the line

```
with open(filename, 'r') as h:
```

with

```
h = open(filename, 'r')
```

That is, do without the `with` functionality. This will make the restructuring a bit simpler. You can close the file with `h.close()`.

3. Rewrite Lab 1 using exceptions so that the program does not crash on improper input.

Chapter 7

Modules, libraries, and program structure

7.1 Modules

Many programming languages come with a “module system”, Python among them. These are useful when one writes larger programs and libraries of one’s own. They can be used for:

- Code reuse – importing code that other programmers (or you yourself) have written;
- Structure – it is hard to work with a lot of code in one file, so spread it over several files instead;
- Collaboration – it is much easier to write code together as a team when it is divided up so that not everyone is making changes in the same file.

With the help of modules, you can begin to understand a large project as composed of independent units. Ideally, you do not need to understand all the details of a module: it is enough to understand its *interface*. The interface consists of the functions and variables intended to be visible to other programmers who use the module.

Large programs quickly become impossible to fully understand at a glance. Well-chosen module structure is thus essential for grasping both your own and others’ programs.

A technical advantage of modules is that they help you to manage *namespaces*. Often interacting programs will want to use the same identifiers for functions and variables (for example sort). Modules conceal implementations as well as variable and function names.

Principles:

- Every Python file is a module.
- Anything to be used from another file must first be *imported*.

7.1.1 Importing modules

We can import modules in a number of different ways:

- `import modulename`: Allows functions and variables to be used with dot notation; in other words, the contents of `modulename` are imported “qualified”.

```
import math
math.ceil(3.5) # Round up
ceil(4.4) # Error!
```

- `from modulename import f1, f2, ...`: Allow use of `f1` and `f2` (but no others) in your program without dot notation.

```
from math import ceil
ceil(3.5)
```

- `from modulename import *`: Make everything from `modulename` accessible without dot notation.

Warning: this is considered to reduce readability and makes it difficult to tell which parts of a module your program depends on.

Obs: identifiers that begin with `_` are not imported. These are considered “private” to the module.

Note: some modules assume that they will be imported using `*`, as they work by overwriting certain built-in functions.

- `import modulename as X`: Use modules with long names more easily with dot notation. For example:

```
import math as M
M.ceil(3.4) # Useful for modules with long names
```

- `from modulename import X as Y`: For avoiding naming collisions. For example:

```
from math import gcd as libgcd

def gcd(a,b):
    r = a % b
    if r == 0:
        return b
    else:
        return gcd(b, r)

for a in range(2,10):
    for b in range(2,a):
        print(gcd(a,b) == libgcd(a,b))
```

- Modules can be grouped in *packages*, which can be understood as hierarchically arranged groups of modules.

```
import os.path as OS
if OS.isfile('data.txt'):
    ...
```

7.1.2 Writing your own modules

Suppose we have the following Python code in a file `circle.py`:

```
pi = 3.14
```

```
def area(radius):
    return pi * (radius ** 2)

def circumference(radius):
    return 2 * pi * radius
```

We can then import it in another file:

```
import circle

print(circle.pi)
print(circle.area(2))

import circle as C

print(C.pi == circle.pi)

tau = C.pi * 2

def circumference(radius):
    return tau * radius

print(C.circumference(2) == circumference(2))
```

Note: the file `circle.py` must be somewhere Python can find it when it looks for modules to import. The first place Python looks is the folder where the program/.py-file you are running is stored; the easiest option is therefore to keep your module file (here `circle.py`) in the *same* folder as the main program/file that imports the module.

7.1.3 `main()`-functions

A common idiom:

```
print("Hello")

def main():
    print("python main function")

if __name__ == '__main__':
    main()

print("__name__ value: ", __name__)
```

Here, `main` is called if and only if one runs this program as a standalone file. If the file is used as a module instead, then `main` is not run. You can thereby write modules that contain example programs and/or test code that is not run when the module is imported. The reason that `main` is not run if the file is imported as a module is that the variable `__name__` is set to the module name in that case, so the body of the `if`-statement is not run.

7.2 Popular libraries and modules

Python comes with batteries included: there are heaps of good and useful modules that come as part of the Python implementation. These modules are called Python's standard library. For a long list and documentation, see <https://docs.python.org/3/library/>.

- `math` – Mathematics: trigonometry, logarithms, et cetera.
- `sys` – Advanced file handling and more.
- `os` – Helps avoid depending on specifics of Win/Mac/Linux (for example avoid problems with “/” in filenames in Linux/Mac vs. “\” in filenames for Windows).
- `argparse` – Used to access parameters when one starts a program from the command line.
- `itertools` – Useful for advanced and fast iteration.
- `functools` – Several higher-order functions.
- `datetime` – Date and time types.
- `pickle` – Serializing of Python objects.
- `csv` – It is harder than you think to read a comma-separated file!
- `json` – Save data in JSON format so that you can easily read it in again.
- `matplotlib` – Visualize data.
- `numpy` and `scipy` – For calculations.
- ...: Investigate for yourself!

7.2.1 JSON and serialization (not on exam)

Problem: it is a pain to write and read files! Especially when it comes to large data structures (think: nested lists and/or dictionaries).

The solution: serialization.

Idea: dump data in a file in a standard format that can easily be read back in again.

There are standard Python-specific modules for this (`marshal` and `pickle`).

A good *portable* solution: JSON – JavaScript Object Notation

```
import json
legends = [ ['Lovelace', 'Ada', 1815]
            , ['Babbage', 'Charles', 1791]
            , ['Beurling', 'Arne', 1905]]

# Create a data file:
with open('legends.json', 'w') as outfile:
    outfile.write(json.dumps(legends))

# Later... write out birth years:
with open('legends.json', 'r') as data_h:
    external_data = json.load(data_h)
```



```
for x in external_data:
    print(x[2])          # Print birth year
```

7.2.2 NumPy: for vectors and matrices (not on exam)

numpy (<https://www.numpy.org/>) is a package for scientific computation in Python. It comes with very efficient storage of vectors and matrices using “arrays”. Python’s list supports typical array operations, but also much more. That flexibility comes at the cost of efficiency. Arrays in numpy are closer to the computer’s own representation, which makes calculations more efficient (better use of memory and computation time). Efficiency comes at the cost of flexibility, but in many applications it is well worth the tradeoffs.

One can create vectors and matrices using numpy arrays:

```
import numpy as np          # It is idiomatic Python to import numpy as np

a = np.array([1,2,3,4])    # From an ordinary list
b = np.ones((2,2))         # 2x2 unit matrix
c = np.zeros((10,10))      # 10x10 zero matrix
d = np.linspace(0, 1, 16)  # 16 elements from 0 to 1, floats!
e = np.reshape(d, (4,4))   # Creates a 4x4 matrix with numbers from 0 to 1

print(a)
print(b)
print(c)
print(d)
print(e)
```

```
[1 2 3 4]
[[1. 1.]
 [1. 1.]]
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
[0.         0.06666667 0.13333333 0.2         0.26666667 0.33333333
 0.4         0.46666667 0.53333333 0.6         0.66666667 0.73333333
 0.8         0.86666667 0.93333333 1.         ]
[[0.         0.06666667 0.13333333 0.2         ]
 [0.26666667 0.33333333 0.4         0.46666667]
 [0.53333333 0.6         0.66666667 0.73333333]
 [0.8         0.86666667 0.93333333 1.         ]]
```

Note that np is an established alias, and import is most often used as in this example, including in the

numpy documentation.

Tip: try for example `help(np.linspace)` to see how it works.

We naturally get access to various operations on matrices, for example matrix multiplication:

```
m = np.array([[1,2], [3,4]])
v = np.array([1, 0.5])
product = np.matmul(m, v)
```

```
[[2. 5.]]
```

numpy generously regards an array as a vector in an appropriate sense. Here, the above `v` is treated as a column vector; if we wrote `np.matmul(v,m)` instead, it would be treated as a row vector.

There is a special data structure for matrices defined in numpy, but it is avoided today as there are plans to remove it. The current recommendation is to always use `np.array`.

numpy is a part of `scipy` (<https://www.scipy.org/>), which contains a number of different packages for doing scientific computation and handling “big” data with Python.

7.2.3 Matplotlib: plotting data

Here we will see some simple approaches to plotting data. We will make use of the sub-module `pyplot` within `matplotlib`. The module `pyplot` contains a collection of functions that make `matplotlib` work roughly like MATLAB.

7.2.3.1 Graphs

The below code plots values in a line plot (function) by connecting the points with a line.

```
from matplotlib import pyplot

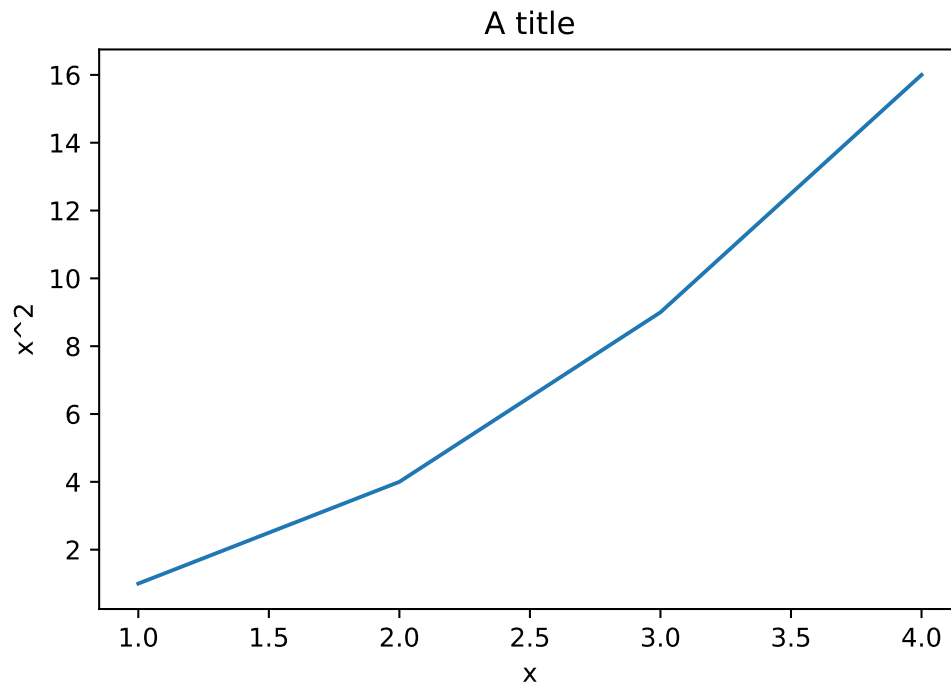
xs = [1,2,3,4]
ys = [1,4,9,16]
pyplot.plot(xs, ys)

pyplot.title("A title")
pyplot.xlabel("x")
pyplot.ylabel("x^2")

# pyplot.savefig("X/graph.pdf") # Save to file. X should be a path, e.g. "/Users/ksahlin/DA7066/".
pyplot.show() # A figure will pop up in VS code.

pyplot.clf() # this clears the figure for any upcoming plots
```

This code produces the following plot:



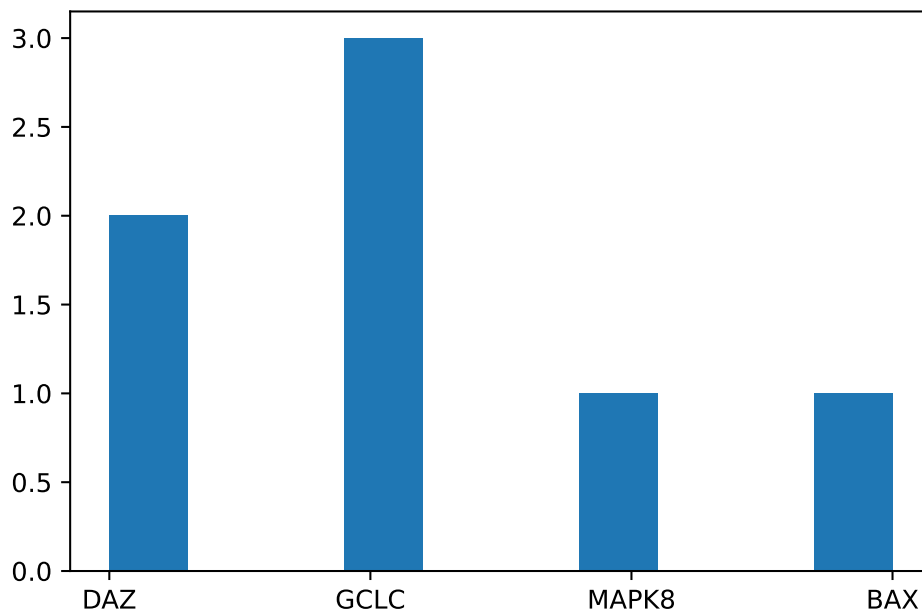
If we want to save the plot, we can use `plt.savefig("X/graph.pdf")` where X is the path on your system, e.g., `/Users/ksahlin/da7066/lecture6/graph.pdf`. If you want to save the plot to file you have to use `plt.savefig` *before* `plt.show`.

7.2.3.2 Histograms

Many types of analysis requires histograms. Consider below example.

```
from matplotlib import pyplot
# most expressed gene in sample, 10 experiments
expressed = ['DAZ', 'GCLC', 'MAPK8', 'BAX', 'GCLC', 'GCLC', 'DAZ']
pyplot.hist(expressed)
pyplot.show()
```

This code produces the following plot:



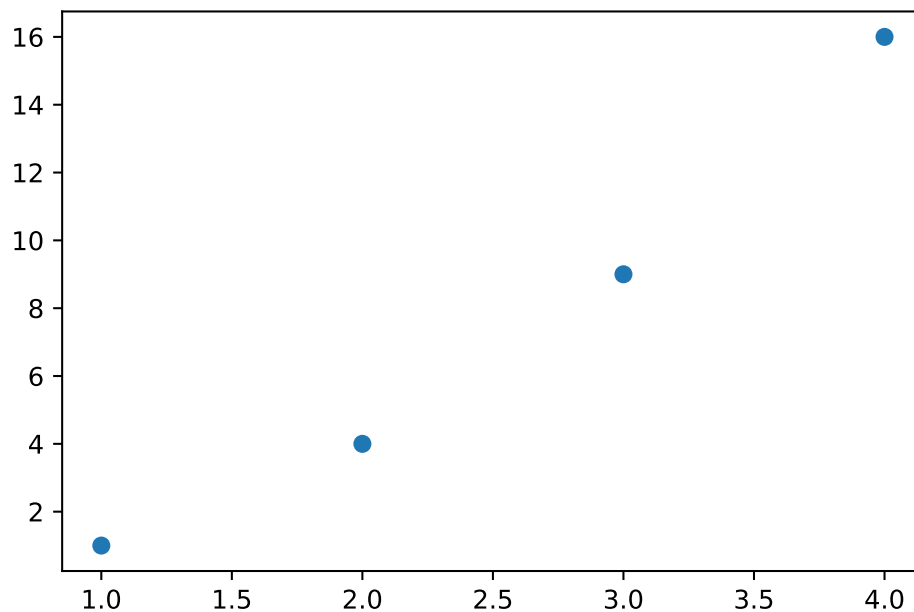
You can also make a histogram of continuous data and `hist` will automatically try to bin the values. Try for example `pyplot.hist(nums)` where `nums = [1,1.5,1,14,4.2,5,5,5,3,3,2,4,9,16]`

7.2.3.3 Scatterplots

Scatterplots can be used to plot individual datapoint from, e.g., experimtn measurments. Lets first look at the scatterplot behaviour when plotting the same data as in the graph example.

```
from matplotlib import pyplot
xs = [1,2,3,4]
ys = [1,4,9,16]
pyplot.scatter(xs, ys)
```

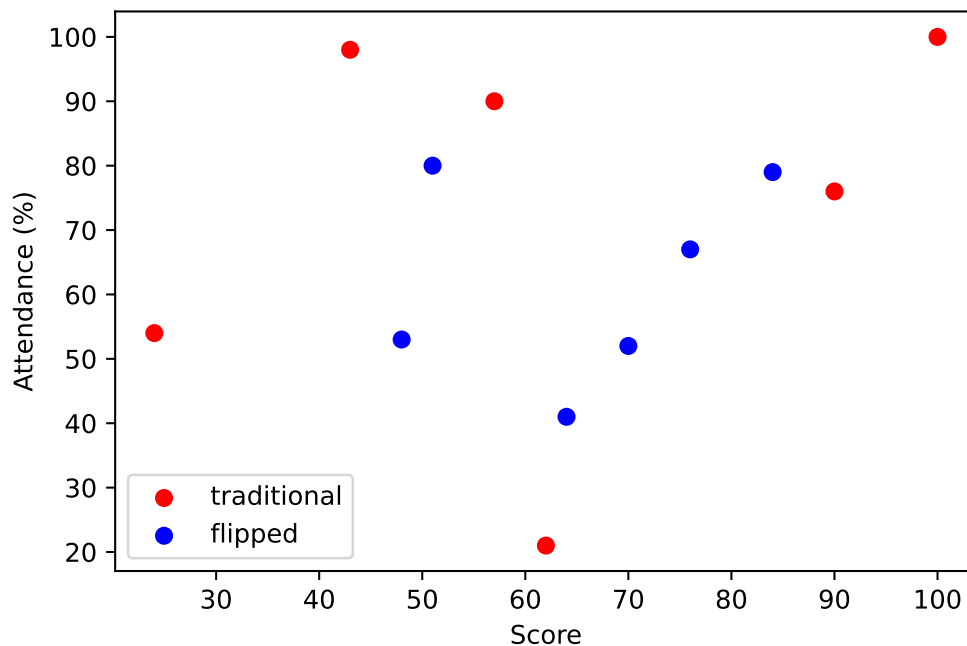
This code produces the following plot:



As we can see the scatterplot simply plots the individual data points. Let's assume we have a set of measurements over two variables (score and attendance) from two classes following two different teaching formats ('traditional' and 'flipped classroom'). We can plot the data as follows.

```
from matplotlib import pyplot
t_scores = [100, 57, 24, 43, 90, 62]
t_attendance = [100, 90, 54, 98, 76, 21]
f_scores = [84, 76, 64, 48, 51, 70]
f_attendance = [79, 67, 41, 53, 80, 52]
pyplot.scatter(t_scores, t_attendance, color='r', label='traditional')
pyplot.scatter(f_scores, f_attendance, color='b', label='flipped')
pyplot.xlabel("Score")
pyplot.ylabel("Attendance (%)")
pyplot.legend()
pyplot.show()
```

Note: These are made up values and not from an actual experiment. This code produces the following plot:



There are many other formats to visualize data with Matplotlib (pie charts, three dimensional plots, fitting regression lines, etc.), feel free to explore this at <https://matplotlib.org/>.

7.2.3.4 Seaborn

Matplotlib is a highly versatile library, though it comes with a somewhat complex interface. Many scientists prefer R's programming plotting library, ggplot2, due to its ease of use. To address this, the nonstandard Python library seaborn (<https://seaborn.pydata.org/>) has been developed, aiming to mimic the user-friendly interface of ggplot2. If you are planning to routinely use Python for data analysis, I recommend taking a look at seaborn.

7.3 Rules of thumb for program structure and good code

It is hard to write good code. By “good” code, we mean code that is easy to read, understand, use, and maintain. By writing good code, we can also avoid introducing bugs. It is difficult to say exactly what makes code good, but hopefully you have gotten some experience of both good and less good code by reviewing each other's code. Many programmers have different definitions of what makes code good, in the same way that “good art” and “good literature” are very subjective.

You can find many Python-specific tips at <https://docs.python-guide.org/writing/style/>. There is also a style guide, with the mysterious name PEP-8, which was written by Guido van Rossum, the creator of Python. When you start programming in another language, it is a good idea to find out which conventions its programmers follow—it differs from language to language.

There are tools for automatic review of code. One of these is <https://www.pythonchecker.com/>, which runs on the web. There are also IDE's with similar support and libraries such as pylint that you can

run in a terminal window.

7.3.1 Programming guidelines

Here is a list (in no specific order) of things that we think make code better:

- Avoid long lines; less than 79 characters per line is good. If a line is too long, it becomes hard to read. [PEP-8](#) specifies this limit.
- One unit of code per line. The following code is bad:

```
print('one'); print('two')

if x == 1: print('one')

if <complex comparison> and <other complex comparison>:
    # do something
```

The following code is much better:

```
print('one')
print('two')

if x == 1:
    print('one')

cond1 = <complex comparison>
cond2 = <other complex comparison>
if cond1 and cond2:
    # do something
```

- Strive to write short functions. The majority of functions should fit in half a laptop screen with font size 12. In some cases, they can of course be longer, but think carefully whether it is possible (or would be clearer) to factor out the code into smaller functions first. When a function is very long, it is hard to get a sense of everything it does at a glance.
- Divide up code into functions that do **one** thing and do it well. This makes it much easier to read, test, and understand the code.

Example: “read in several datasets, do a calculation with each one, and write out a summary” should be divided up like so:

- One function for reading in datasets.
 - One function that does the calculation on a dataset.
 - A main function that calls the two other functions and writes out the summary.
- Functions should return something. That way it is easier to test them and to understand what they do or are expected to do. There are situations where a function doesn’t *need* to return anything (printing, list manipulation, and so on), but the default expectation should be that a function returns something. A common beginner mistake is to use global variables to handle data flow in a program and thus “not need” return values. Among beginners, a lack of return values can also be an indication of bad program structure.
- Every function should have a documentation string.

- Separate **calculation** from **interaction**: functions that do calculations should not use `print/input`. Functions that handle interaction should not do complex calculations. By interaction, we mean simple `print` statements as well as questions to the user. You will often want to reuse calculating functions without any associated interactive components.
- Think about how a program should be structured and which abstractions it should use *before* you start to code. Group functionality and attributes in classes¹—so as to maximize abstraction and it will be easier to change your code later.
- Divide up code into small units, that is, into appropriate functions and classes. Code is called *modular* when every logical component of the code can be studied as its own little module which is then combined with other modules to create complex programs. This makes the program much more easily understandable.
- Do **not** return values of type `int` or `bool` as strings, for example `'2'` or `'True'`. This makes it harder to test and use functions later. It is also easy to introduce bugs in this way. For example:

```
def f(x):
    return '2'

if f(1) == 2:
    print("This is not printed!")
else:
    print("This is printed!")
```

- Do not return “magic” values to indicate errors or problems. Use exceptions instead. For example: “`return 1000000`” is a bad way to signal “does not converge”. **Exception:** `None` can be an acceptable magic value.
- Multiple return values are practical! Better than strings that contain several values.
- Variables/constants are better than hard-coded literals. For example, if we write a graphics program and want a window with size `600 x 600`, then it is better to first write

```
width , height = 600 , 600
```

and then then use `width/height` instead of repeating `600` everywhere. This makes it easier to change the program in the future if we want the window to have a different dimension. It is also easier for someone reading code to understand our intention when we use `width` as opposed to some arbitrary number.

- Avoid global variables, as these often lead to “spaghetti code”: messy code where completely separate parts of the program can affect each other. A reader should be able to understand one function without looking at other parts of a program. It is especially dangerous to manipulate global variables in functions (by using the keyword `global`).
- Include tests to both test and specify how a program is expected to behave. This is easier to do if the program is divided into smaller functions that do **one** thing and **return** it.
- Simple code is easier to read. You should therefore avoid writing unnecessarily complicated code. For example, “`if b == True:`” is more complicated than just “`if b:`”. It is like Swedish: it is bad style to staple many small words together. In the same way that simple sentences are easier to read, simple code is easier to understand.

¹Classes will be introduced in ??.

Here is an example of an unnecessarily complicated program:

```
def make_complex(args):  
    x, y = args  
    d = dict()  
    d['x'] = x  
    d['y'] = y  
    return d
```

Much better:

```
def make_complex(x,y):  
    return {'x': x, 'y': y}
```

A good trick is to always read through your code several times and consider how it might be simplified.

- Be careful with mutable datatypes—many bugs arise from lists being accidentally emptied by say `pop()`! It is especially dangerous to store mutable objects in global variables or class attributes.
- Divide up larger projects in logical units in different files/folders.
- It should be clear from the identifier what a variable contains or what a function is meant to do. For example, if you have the line

```
x = 17 # critical value, abort if exceeding
```

then it is naturally better to replace `x` with something else, for example:

```
critical_value = 17 # abort if exceeding
```

One more example: instead of

```
nc = 1.7 # normalization constant
```

it is better to write:

```
normalization_constant = 1.7
```

Now no commentary is necessary.

- There are situations where function names can be uninformative: for example, the purpose of `main` is clear by convention.
- Avoid generic identifiers. Use identifiers that are descriptive.
 - *Bad*: `start`, `compute`, `f`, `foo`
 - *Good*: `remove_outliers`, `newton_raphson`, `compute_integral`
- Code changes. If a function's purpose changes, then you should change its name! It is confusing if your function is called `print_integral` but doesn't print anything.
- Comments are good, but not a replacement for simple code with well-chosen identifiers. If your code needs comments to be understandable, then you are probably doing something complicated. Good uses of comments are to explain why you are using a specific data structure (for example, `set` instead of `list`) or how a function is expected to be used.

Comments that explain nothing more than what the code already says should be avoided. For example, the following comment is completely redundant:

```
rows = []          # Create an empty list
```

Anyone who knows a little Python already understands that this line creates an empty list, so why write it in a comment?

7.3.2 Signs of not-so-good code

- “Copy-paste code”: it is easy to introduce bugs when you copy lines and paste them again elsewhere. The program also gets hard to read if it gets too long. Can you introduce a loop or function instead?
- Is it hard to come up with good tests? Then you haven’t divided up your program into small parts that do **one** specific thing. Rethink your design and introduce better functions and abstractions.
- Deep indentation: if you have deeply nested **if**-statements, the code becomes hard to read and should be divided up into smaller parts. A common cause of deep indentation is that you have several different logical cases; you should then create functions for the different cases. In this way, the case analysis becomes clearer, with the different function calls indicating what happens in each case.

Here is an example of “bad” code:

```
# Print the prime numbers from a collatz sequence
# for 100 consecutive starting numbers
for i in range(1,100):
    n = i
    primes = []
    while n != 1:
        is_p = True
        for x in range(2, n):
            if not (n % x):
                is_p = False
        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    print("Start number :", i, "Primes in Collatz sequence: ", primes)
```

This code block is indented to four levels. It isn’t terrible, but we can structure it better:

```
def collatz_primes(n):
    """Computes prime numbers from
       the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    list: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        is_p = True
```

```

        for x in range(2, n):
            if not (n % x):
                is_p = False
        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

We have broken out a special function, `collatz_primes(n)`, which is used in a `for`-loop, and the function has only three indentation levels (the `while`-loop, the `for`-loop, and the `if`-statement).

We can go a step further and move the update of the variable `n` and primality test into their functions:

```

def collatz_update(n):
    if n % 2 == 0:
        return n//2
    else:
        return 3*n + 1

def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True

    for y in range(2,n):
        if not (n % y):
            return False
    return True

def collatz_primes(n):
    """Computes prime numbers from
       the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    list: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        if is_prime(n):
            primes.append(n)

```

```

        n = collatz_update(n)
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Which version do you think is easiest to read and understand?

- Do your functions that do calculations lack parameters and **return**-statements? This will likely cause you trouble. The purpose of a function is usually to take inputs and compute something. The computation need not be mathematical—it can simply be manipulation of data. It is less common that a function lacks parameters entirely, but there are of course exceptions, for example a function that prints a menu or initializes a data structure. For beginners, however, it is often tempting to store data in one or more global variables that functions later work with. Such a setup makes it difficult to see how functions depend on each other; you need to read functions carefully to identify the flow of data through the program. And you cannot use the function in another module later on if it doesn't take its parameters explicitly!

The code below works even if we put `is_prime` in another module `m` (that is, another file) and call it with `m.is_prime(x)`:

```

def is_prime(x):
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is prime:", is_prime(x))

x = int(input("Provide a number to analyze: \n"))
main(x)

```

The code below is more poorly written and works only if we have the global variable `x` in the same file. This makes it impossible to usefully import the function in another file.

```

def is_prime():
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is prime:", is_prime())

```

```
x = int(input("Provide a number to analyze: \n"))
main(x)
```

7.4 Exercises

1. Import the library `random` (see <https://docs.python.org/3/library/random.html>) and use it to generate 10 random *integers* between 1 and 100 and 10 random *floats* between 1 and 100.

Tip: Read the documentation. Which functions are the most appropriate in this case?

2. Consider the code below, which reads in an integer and decides if it is a perfect number or a prime number. Rewrite the code so that it has better structure.

Tip: Divide up the code into functions. Which parts of the code can naturally be made into functions? What should the functions take as parameters and return so that they can be useful outside this module? You might want to have a main function `main(x)` that takes the number `x` as a parameter.

```
# checks if a number is (1) perfect (2) prime
x = int(input("Provide a number to analyze: \n"))
sum = 0
for i in range(1, x):
    if (x % i == 0):
        sum = sum + i

if x >= 2:
    prime = True
    for y in range(2,x):
        if not (x % y):
            prime = False
else:
    prime = False

if sum == x:
    print("The number is perfect")
else:
    print("The number is not perfect")
if prime:
    print("The number is prime")
else:
    print("The number is not prime")
```

3. Adjust your restructured program from Exercise 2 to let the user decide whether to choose a number themselves or let the program choose one at random. Where in the program should we put this new code?
4. The code below reads in a DNA-string (for example 'AGCTAGCGGTAGC') and first looks for the most commonly occurring substring of length `k`. Then the program prints out the pairwise distances between copies of this string. For example, for 'AGCTAGCGGTAGC' and `k = 3`, the code below returns [4, 6]: there are 4 and then 6 nucleotides separating the start positions of the three

occurrences of the most common substring, 'AGC'.

Exercise: Rewrite the program to be more readable. Which parts can be made into sensible functions? Make sure they both take appropriate parameters and return appropriate data structures. It should be possible to call any of the functions individually from another module.

```
# Given a sequence as input find distances between the copies of
# the most frequent substring

seq = input("Enter a dna string: ")
k = 3
substrings = {}
for i in range(len(seq) - k + 1):
    substring = seq[i:i+k]
    if substring in substrings:
        substrings[substring] += 1
    else:
        substrings[substring] = 1

max_count = 0
most_freq_substring = ""
for substring, count in substrings.items():
    if count > max_count:
        most_freq_substring = substring
        max_count = count

positions = []
for i in range(len(seq) - k + 1):
    substring = seq[i:i+k]
    if substring == most_freq_substring:
        positions.append(i)

distances = []
for p1, p2 in zip(positions[:-1], positions[1:]):
    distances.append(p2 - p1)

print(distances)
```

5. † You have five numbers in a list [13, 24, 42, 66, 78] and want to calculate all the ways that a number can be written as a “direct concatenation” of two elements from the list; by a “direct concatenation” of two integers we mean for example 13, 78 -> 1378 and 66, 42 -> 6642. That is, want to look at all permutations of two elements from the list and combine each permutation to make a number.

Tip: Look in the standard module `itertools` for permutations. To concatenate the integers, try first explicitly coercing the numbers to `str`, concatenating them with the string function `.join()`, and finally coercing them back to `int`.

The output, if all the numbers are saved in a list that is then printed, should be:

```
[1324, 1342, 1366, 1378, 2413, 2442, 2466, 2478, 4213, 4224, 4266, 4278, 6613, 6624, 6642, 6678, 7813, 7824, 7842, 7866]
```

6. Rewrite lab 1 (temperature conversion) so that the functions are in a separate module with the name `temp_functions.py` and document each function with a documentation string (see chapter 2). The main program should import the functions in `temp_functions.py` and handle errors with **try/except**.

Chapter 8

Writing a command line tool, testing

8.1 The terminal

8.1.1 Why?

So far in this course, we have simply pressed the “run” button on our code named, say `my_code.py`, and VS code has automatically run `python my_code.py` in the terminal in a window below the code. This is fine, but the terminal supports more advanced usage. In fact, you can think of the terminal as a generic *environment* to write and execute programs.

Many bioinformatic analyses pipelines use the terminal to automate different processing steps of data. These processing steps might be performed by different programs, each optimized for their task. Furthermore, the programs may be developed in different programming languages, and some may be available only in binary format.

Let us consider a concrete example. Assume you have obtained Illumina reads from a species without a reference genome, and your task is to assemble the genome and to find gene loci. Consider the steps we have to perform to go from Illumina reads to predicting a non-model organism’s gene loci. First, we need to trim away the adapters in the reads, then we need to assemble the reads into chromosomes, and finally, we can use a script that (let’s assume) we wrote to find genes.

The programs to trim adapters (e.g., Cutadapt) and to assemble reads (e.g., SPAdes) may be binary code (written in C/C++) while our program `find_genes.py` is in Python. Our *pipeline* will look like:

```
Cutadapt reads.fa trimmed_reads.fa
SPAdes trimmed_reads.fa assembled_genome.fa
find_genes.py window_size assembled_genome.fa gene_regions.csv
```

This pipeline can be run in the terminal through a *shell script*. We will not cover shell scripting in this course, but you can think of it (simplistically) as a programming language to “connect” programs. In chapter 8 we will see examples of *pipelining* which is a simple command to connect input and output between programs.

For this pipeline to work, our script `find_genes.py` needs to read arguments from the terminal. Our script takes three *arguments*. The `window_size` parameter is the size of a segment where we estimate

the GC content¹. The `assembled_genome.fa` is a fasta file output from SPAdes, and `gene_regions.csv` consists of three columns representing chromosome, start, and stop coordinate where each row is a “gene region”.

The benefit of putting together programs this way is that the above pipeline can be executed overnight without manual oversight. You will have the results in the output file the next day **without manually starting each program**. This is one of the beauties of making your programs executable from the terminal. The above pipeline is a toy example. In real scenarios, pipelines may contain tens to hundreds of steps, have cyclic execution patterns (repeating the pipeline until convergence), and other complicated procedures. In such scenarios, you do not want to be at the computer each time a task is ready, and the next one needs to start.

8.1.2 Running a Python program from terminal

You have already accessed the terminal through VS Code many times in this course. It is accessed by either pressing the “run” button in VS Code, or opening the *Command Palette* in VS Code and typing “Terminal” and press enter (see Chapter 1). The terminal in VS Code is automatically placed in the same folder as your current `.py` program.

You can also start the terminal window “outside” of VS Code. You can easily search on the web how you do this on your computer system “open terminal window on [your OS]”. Below are some common systems:

- Open the terminal
 - Mac/Linux: search for “terminal” in “spotlight” and open it.
 - Windows: It is called the “Command Prompt”. Click “Start”, then “All Programs”, then “Accessories” then click “Command Prompt”)

If opened outside of VS Code, you need to navigate to the folder that you have your Python scripts in, using `cd` (change directory).

- navigate to the directory of the Python file:
 - Mac/Linux: use `cd /Users/UserName/my_files/`
 - Windows: use `cd \Users\UserName\my_files\`

Assume you have the unfinished code below in a file `find_genes.py` placed in a folder on your computer `/path/to/your/folder/find_genes.py`.

```
def main():
    print("This is the start of my find_genes program")

if __name__ == '__main__':
    main() # Call the main function if run from terminal
```

To run a program from the terminal, we can now run the file from the terminal window as: `python find_genes.py` and it will print the statement. You can also verify that the print statement will not be printed if you import this file as a module, e.g., with `import find_genes` into another script. This is controlled through the *special variable* `__name__`, as we discussed in chapter 6. The `__name__` variable will be set to `find_genes` if you have imported the script with `import find_genes`, and to `__main__` if you are running it as the main script.

¹We will here assume that genes can be found simply from observing regions having a GC content over 70%. This is obviously not true but simplifies our example.

8.2 Reading arguments from terminal

8.2.1 Using sys module

We will now make our script be able to take three arguments, namely the window_size (integer), an input file assembled_genome.fa (file object), and an output path gene_regions.csv (file object). We parse such basic arguments from the command line with the module sys as follows.

```
import sys

def main(w, assm_file, outfile):
    print("The arguments I read were:")
    print(w)
    print(assm_file)
    print(outfile)

if __name__ == '__main__':
    w = int(sys.argv[1])
    assm_file = open(sys.argv[2], 'r')
    outfile = open(sys.argv[3], 'w')
    main(w, assm_file, outfile) # Call the main function if run from terminal
```

The sys.argv is a list with strings, where the strings are the text occurring after the python command in the terminal. Create an dummy file test_file.fa (can be empty for now) and test the code as:

```
python find_genes.py 50 test_file.fa outfile.csv
```

If we print the sys.argv variable in our script, we will get

```
['find_genes.py', '50', 'test_file.fa', 'outfile.csv']
```

As you can see, at position 0 in the list is the script name find_genes.py. We extract the arguments from the list from position 1 and onwards as shown in the code and convert them to the suitable data types.

What happens if you give too many arguments? Too few arguments? There will be a cryptic error message!

8.3 Argparse module

The sys module is a very rudimentary module to handle commandline options. Let's instead use the argparse module. Full documentation here: <https://docs.python.org/3/library/argparse.html>. Let's update our code to use argparse.

The updated find_genes.py will look as follows

```
import argparse

def main(args):
    print("The arguments I read were:")
    print(args.w)
    print(args.assm_file)
    print(args.outfile)
```

```

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Find gene region \
                                         in a genome assembly through GC content.", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('w', type=int,
                        help='Window size to calculate GC content from')
    parser.add_argument('asm_file', type=argparse.FileType('r'),
                        help='input assembly file')
    parser.add_argument('outfile', type=argparse.FileType('w'),
                        help='Output CSV-file')
    args = parser.parse_args()
    main(args)    # Call the main function if run from terminal

```

While there is a little bit more code in this example, the `argparse` module will do a lot for us. We have for example specified the types of the arguments upon reading, which means we do not manually need to convert their type. Also, if we run the script with too few arguments

```
python find_genes.py 50 test_file.fa
```

we will get a nice message about the usage

```
usage: find_genes.py [-h] w asm_file outfile
```

and the same if we run with too many arguments

```
python find_genes.py 50 test_file.fa outfile.csv one_more_arg
```

```
usage: find_genes.py [-h] w asm_file outfile
find_genes.py: error: unrecognized arguments: one_more_arg
```

You can also type `--help` to get a nice help message based on what you have specified in the help parameters in the code

```
python find_genes.py --help
```

```
usage: find_genes.py [-h] w asm_file outfile
```

Find gene region **in** a genome assembly through GC content.

positional arguments:

```

w           Window size to calculate GC content from
asm_file    input assembly file
outfile     Output CSV-file

```

optional arguments:

```
-h, --help  show this help message and exit
```

8.3.1 Adding features to our program

Another benefit of the `argparse` module is that it handles more types of arguments (e.g., required, optional), can set default values to arguments, and more. Assume we have the program `find_genes.py`

implemented already. Let's assume our collaborator is happy with our program but wants the following updates:

- Make the `window_size` an optional parameter, since most of the time, people use, e.g., 50.
- The assembly file only provides strands in one (arbitrary) direction. The user might want to find genes using also the GC-content on the reverse complement strand. Add this as default behaviour, with the option to ignore reverse compliments.
- Add more diagnostic output to the user (e.g., which step or chromosome the program is currently working on)

Below shows the added arguments needed in the `find_genes.py` file based on the requests of our collaborator.

```
import argparse

def main(args):
    print("The arguments I read were:")
    for arg, val in vars(args).items():
        print(arg, val)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Find gene region \
                                         in a genome assembly through GC content.", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('--w', type=int, default = 50,
                        help='Window size to calculate GC content from')
    parser.add_argument('asm_file', type=argparse.FileType('r'),
                        help='input assembly file')
    parser.add_argument('outfile', type=argparse.FileType('w'),
                        help='Output CSV-file')
    parser.add_argument('--use_rc', action="store_true",
                        help='Use also reverse complements to find genes')
    parser.add_argument('--verbose', action="store_true",
                        help='Print diagnostic output to terminal (stdout)')
    args = parser.parse_args()
    main(args) # Call the main function if run from terminal
```

In this example, `asm_file` and `outfile` are *required* arguments and `w`, `rev_comp`, and `verbose` are optional arguments.

Notice that we have changed the printing of arguments to a for-loop as the list of arguments is getting longer. The `vars()` function is a special function in python that returns the `__dict__` attribute of an object. The `__dict__` attribute is a dictionary containing the object's changeable attributes.

We can see all the parameters and their values if we run the script. Notice the change in specifying the window size.

```
python find_genes.py --w 30 test_file.fa out.csv
```

```
The arguments I read were:
('w', 30)
('asm_file', <_io.TextIOWrapper name='test_file.fa' mode='r' encoding='UTF-8'>)
('outfile', <_io.TextIOWrapper name='out.csv' mode='w' encoding='UTF-8'>)
```

```
('use_rc', False)
('verbose', False)
```

We can change the arguments we specify of the optional parameters.

```
python find_genes.py test_file.fa out.csv --use_rc --verbose
```

The arguments I read were:

```
('w', 50)
('asm_file', <_io.TextIOWrapper name='test_file.fa' mode='r' encoding='UTF-8'>)
('outfile', <_io.TextIOWrapper name='out.csv' mode='w' encoding='UTF-8'>)
('use_rc', True)
('verbose', True)
```

8.3.2 Complete code

Below shows the full program

```
import argparse

def read_fasta(f):
    seqs = {}
    for line in f:
        if line[0] == ">":
            acc = line.strip()
            seqs[acc] = ''
        else:
            seqs[acc] += line.strip()
    return seqs

def get_kmer_windows(seq, w):
    return [seq[i:i+w] for i in range(len(seq) - w + 1)]

def estimate_gc(kmers):
    return [(kmer.count('C') + kmer.count('G')) / len(kmer) for kmer in kmers]

def get_gene_regions(gc_fractions):
    regions = []
    curr_region_start = 0
    for i in range(len(gc_fractions) - 1):
        # gene region starts
        if gc_fractions[i] < 0.7 and gc_fractions[i+1] >= 0.7:
            curr_region_start = i+1

        # gene region is over
        elif gc_fractions[i] >= 0.7 and gc_fractions[i+1] < 0.7:
            regions.append( (curr_region_start, i+1) )

    return regions
```

```

def print_results(chr_id, regions):
    for start, stop in regions:
        print(chr_id, start, stop)

def main(args):

    assembly = read_fasta(args.assm_file)

    for chr_id, seq in assembly.items():
        kmers = get_kmer_windows(seq, args.w) # returns list of k-mers in order
        gc_fractions = estimate_gc(kmers) # estimates GC content per kmer, returns list of floats
        regions = get_gene_regions(gc_fractions)
        print_results(chr_id, regions)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Find gene region \
                                         in a genome assembly through GC content.", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('--w', type=int, default = 50,
                        help='Window size to calculate GC content from')
    parser.add_argument('assm_file', type=argparse.FileType('r'),
                        help='input assembly file')
    parser.add_argument('outfile', type=argparse.FileType('w'),
                        help='Output CSV-file')

    # Below args not implemented in this example (see exercise 2)

    # parser.add_argument('--use_rc', action="store_true",
    #                     help='Use also reverse complements to find genes')
    # parser.add_argument('--verbose', action="store_true",
    #                     help='Print diagnostic output to terminal (stdout)')
    args = parser.parse_args()
    main(args) # Call the main function if run from terminal

```

Consider the following file test_file.fa

```

>chr1
AACGTCGCTATAACCGTCGTTATC
>chr2
TTACTCGCACCGTAATCATTCGGTGCT

```

```
python find_genes.py test_file.fa out.csv --w 4
```

Will result in the output

```

>chr1 2 6
>chr1 12 16
>chr2 3 10
>chr2 19 23

```

Below shows only inferred gene regions in capital letters

```
>chr1
aaCGTCGctataACCGTcgttatc
>chr2
ttaCTCGCACGgtaatcatTCGGTgct
```

8.4 Testing

It is challenging to guarantee the correctness of one's code. Mathematical reasoning about code and algorithms can still lead to errors. Regardless of how much planning, thought, and review one performs, it is essential to meticulously test the code. But how is this best done?

Experience shows that it is neither sufficient nor practical to test if an entire program, the actual product, functions as intended. To detect issues early and safeguard against errors due to code changes, testing the program's functions and methods is crucial.

A common approach is to (1) write some code, perhaps a function, (2) verify that the code works, and then continue writing more code. Verifying the code can be straightforward: choose some parameters for the function and check if the result aligns with expectations. A scenario even more prevalent in practice involves:

1. Modify existing code to add new functionality.
2. Verify the new functionality.
3. Verify the old functionality.

Point 3 is easily overlooked but can be challenging to manage. The focus is on the new functionality, not the old, and recalling how the code was tested becomes an issue. When changing a function `func`, do you also remember to test other parts of your code dependent on `func`? As time passes and a project grows, verifying old functionality becomes increasingly difficult. When working with others' code, figuring out how to test existing functions becomes even more challenging. So, how does one address this?

What is needed is systematic and automated testing, achieved by coding the testing, and this methodology is called unit testing. Today, there is excellent support for testing in common programming languages, making it often straightforward to include tests in a project. In this section, we explore various ways to work with tests.

You can read more at: <https://realpython.com/python-testing/>

8.4.1 Testning with assert

Code testing can be done both manually and automatically. For simple manual testing, we can use the `assert` statement in Python.

The **`assert`** condition instruction specifies that the expression condition is expected to be true. If the expression is true, nothing happens, as it is the expected behavior. However, if it is false, the program aborts using a special case, **`AssertionError`**. This type of instruction exists in many languages and serves as a tool for debugging, development, and maintenance of programs.

Some examples:

```

assert len(data_list) > 0
assert x != 0
assert x > 0 and y > 0

```

These lines pose three possible requirements on the state of a program. The **assert** instruction should be interpreted as “this holds.”

8.4.1.1 Testning find_genes.py with assert

Let’s assume you wanted to write some tests for the program find_genes.py that we discussed above. Typically you put the tests in a separate file, e.g., test_find_genes.py.

Below is an example of a partly written test file test_find_genes.py for our program find_genes.py. Note that, for the below script to work, the two files need to be in the same folder².

```

import find_genes
seq1 = 'AGCTGGCT'
seq2 = 'GGTC' #Unexpected short seq
seq_empty = '' # dont forget corner cases
w_3 = 3
w_zero = 0 # Bad user parameter
w_long = 5 # Longer than seq2
w_type = '2' # something else unexpected

assert find_genes.get_kmer_windows(seq1, w_3) == ['AGC', 'GCT', 'CTG', 'TGG', 'GGC', 'GCT']
assert find_genes.get_kmer_windows(seq1, w_long) == ['AGCTG', 'GCTGG', 'CTGGC', 'TGGCT']
assert find_genes.get_kmer_windows(seq2, w_long) == []
# ... ev. more suitable tests here...

kmers = ['AGCTG', 'GCTGG', 'CTGGC', 'TGGCT']
kmers2 = ['AGC', 'GCT', 'CTG', 'TGG', 'GGC', 'GCT']
kmers_empty = [] # dont forget corner cases

assert find_genes.estimate_gc(kmers2) == [2/3, 2/3, 2/3, 2/3, 1.0, 2/3]
assert find_genes.estimate_gc(kmers) == [0.6, 0.8, 0.8, 0.6]
assert find_genes.estimate_gc(kmers_empty) == []
# ... are there more suitable tests?

# If there more suitable functions to test in find_genes.py
# you would add them here...

```

You can execute this file from the terminal to run the tests. Notice that, because get_kmer_windows and estimate_gc are functions that takes arguments and returns data, they are easy to test (recall previous chapter on good programming structure). A sign that you need better structure in your code is if you have problems writing tests such as the ones above. Of course, not all functions are easy or needs to be tested.

²Alternatively, the ‘find_genes.py’ file needs to be ‘found’ elsewhere on the system by Python, e.g., by setting the PYTHONPATH variable in your unix environment

8.4.2 Other methods for unit testing

The `unittest` module has been part of Python's standard library since version 2.1 and is utilized by many projects, both commercial and open-source. To use `unittest`, we need to import the `unittest` module. However, to use the `unittest` module we also need to know a bit about Object Oriented Programming, which is not part of this course. Therefore, the `unittest` module is not part of this course. Feel free to read more about the `unittest` module at <https://docs.python.org/3/library/unittest.html>.

8.5 Exercises

1. The GC fraction is hard coded to 0.7 in the program `find_genes.py` discussed above. Make this an optional argument `--gc_fraction` with default value of 0.7 using the `argparse` framework. What is a suitable datatype for `gc_fraction`?
2. Also implement `--use_rc` and `--verbose` to our program `find_genes.py`.
3. Add **try-except** to your `find_genes.py` program to handle errors in parsing the fasta file from, e.g., malformed fasta headers. See an example below. The program should exit with an error message about the fasta file being malformed. Where would make sense to implement the try-except block?

Case 1 (malformatted)

```
@chr1
CGCTCCTCGT
>chr2
AGCGCTCGT
```

Case 2 (missing)

```
CGCTCCTCGT
>chr2
AGCGCTCGT
```

```
>chr1
AACGTCGCTATAACCGTCGTTATC
>chr2 #no data was observed
```

4. Modify the program below to work from the command line by using `argparse`. The program should take the number `x` (see code below) as a required argument.

```
def is_perfect(x):
    """Checks if an integer x is perfect. That is
    if the sum of divisors to x is equal to x.
    Example of a perfect number is 6, because
    1, 2, and 3 divides 6 and 1+2+3=6.

    Parameters:
    x (int): input number

    Returns:
    bool: True if x is perfect
    """
```

```

sum = 0
for i in range(1, x):
    if(x % i == 0):
        sum = sum + i
if sum == x:
    return True
else:
    return False

def is_prime(x):
    """Checks if an integer x is a prime number. That is
    if x is only divisible with 1 and itself.

    Parameters:
    x (int): input number

    Returns:
    bool: True if x is a prime number
    """
    if x >= 2:
        for y in range(2,x):
            if not ( x % y ):
                return False
    else:
        return False
    return True

x = int(input("Provide a number to analyze: \n"))
print("The number is prime:", is_prime(x))
print("The number is prime:", is_perfect(x))

```

5. Upgrade the program in exercise 4 so that it takes a list of arbitrary many numbers (using argparse) and performs the functions on each of the numbers.
6. Add Unit tests with assert for the program in exercise 5 (see section 7.4.1).

Chapter 9

Unix and the Terminal: Basic commands

9.1 Unix

Using the commandline on Unix-systems There are many Unix systems!

- Linux (with many different distributions)
- NetBSD (BSD: Berkeley Software Distribution)
- OpenBSD
- FreeBSD
- MacOS – derived from (Free?)BSD
- Solaris
- and many more (including Android)

Common notation: `_nix_`

It is the OS of choice for almost everyone around me. All (?) supercomputers are using some `_nix_` dialect.

A guess regarding its popularity:

- The system is a development environment. Great for programmers – and data scientists.
- Principles suggested by its creators:
 - Basic commands designed to do one thing well. – Commands “compose” to more complex tasks. The app situation on mobiles is the opposite of this idea! – “Everything is a file”. A poorly executed principle, but basic idea lives on. Examples:
 - * `/dev/random`
 - * `/proc/cpuinfo` in linux

If you do not have a unix machine, you can work on a virtual unix environment such as:
https://www.tutorialspoint.com/linux_terminal_online.php

9.2 Some commands

Here is a non-exhaustive list of commands that I commonly use regularly:

9.2.1 Navigation

- `cd directory`: Change current working directory. Example: `cd Documents`
- `pwd`: Print the current working directory.
- `mkdir directory`: Create a new directory.
- `ls path`: List files in the specified directory. Use `-l` for detailed information. Example: `ls -l /path/to/directory`

9.2.2 Creating or modifying files and folders

- `touch file.txt`: Create an empty file.
- `cp source destination`: Copy files or directories. Example: `cp source.txt destination/`
- `mv source destination`: Move or rename files or directories. Example: `mv oldfile.txt newlocation/`
- `rm`: remove a file (add `-r` to remove a folder with contents): **WARNING**: do not use this command unless you are absolutely sure you want to remove the file or folder.

9.2.3 Information about files

- `wc -l file`: Count words in a file. Use `-l` to count lines. Example: `wc -l document.txt`
- `cat file`: Display the contents of a file.
- `grep file`: Searches for patterns/strings in files. Example: `grep "error" file.txt` finds lines containing the word "error" in file.txt.
- `head file`: Display the beginning of a file. Use `-n` to specify the number of lines. Example: `head -n 10 file.txt`
- `tail -n file`: Display the end of a file. Use `-n` to specify the number of lines. Example: `tail -n 5 file.txt`
- `cut -c columns file`: Cut out specified columns from a file. Example: `cut -c 1-5 data.txt`
- `sort file`: Sort or merge lines of text alphabetically (add `-n` for sorting on numeric value). Example: `sort data.txt`
- `uniq file`: Display unique rows from a sorted file. Example: `uniq sorted_data.txt`

9.2.4 Misc

- `echo text`: Display text on the terminal. Help: `echo "Hello, World!"`

To see the interface of these commands, we can type `man`, e.g., `man ls`, and it will describe useful arguments. There are many more useful commands.

9.3 Running programs in background (&)

Consider this infinite program in a file `sum.py`:

```
i = 0
while True:
    i+= 1
```

- Example: `'python sum.py &'`
- Commands for checking current programs: `'jobs'/'ps'/'top'`
- To kill: `'kill PID'`

9.4 Signals:

- `'Ctrl-C'`: signal which cancels or terminates the currently-running program
- `'Ctrl-Z'`: Temporarily stop program (send to background)
- `'fg [JOBID]'` restart the job
- `Ctrl-L` : Clear the output of last command
- `Ctrl-K` Clear the terminal screen.

9.5 Composing command lines

- `mkdir x; cd x; mv ../bla.txt .; echo 'Done'` (Continues even if one command fails)
- `mkdir x && cd x && mv ../bla.txt . && echo 'Done'` (Stops at the command failing)

9.6 Streams

- `stdin` (standard input): where your tool/program can read from
- `stdout` (standard output): where your tool/program can write output to
- `stderr` (standard error): where your tool/program can complain

9.6.1 Example 1

```
import sys
x = input('Your name:')
print('Hi', x)
print('bye', file=sys.stderr)
```

Enter the contents above in a file named `example1` running this program as

- `python example1.py`
- `python example1.py > stdout.txt`
- `python example1.py > stdout.txt 2> stderr.txt`

The `>` sign means to redirect all output written to `stdout` to this stream, the `2>` for all output written to `stderr`.

9.6.2 Example 2

```
import sys
import random

i = 0
while True:
    n = random.randint(0,1000000)
    try:
        res = i / n
    except ZeroDivisionError:
        print('Warning: Tried divide {0} with 0.'.format(i), file=sys.stderr)

    if i % 1000000 == 0:
        print('Progress {0} million iterations.'.format(i))
    i+=1
```

Run as - `python example2.py - python example2.py > stdout.txt - python example2.py > stdout.txt 2> stderr.txt`

9.6.2.1 Example 3

```
import sys

for line in sys.stdin:
    n1, n2 = line.split(',')
    print(int(n1)+int(n2))
```

Lets say we have the following data file (data.txt):

```
1,454
9,34
5,54
1,54
3,565
32,23
5,565
9,34
11,34
1,4
1,23
```

Run as - `python program2.py < data.txt - python program2.py < data.txt > outdata.txt`

9.7 Pipes

Unix pipes, also known as pipelines, facilitate the transfer of data between commands. They use the pipe symbol (|) to connect the output of one command as input to another. This method of linking commands enables the creation of powerful and flexible command-line workflows.

9.7.1 Example 1

```
cat logfile.txt | grep "error" | wc -l
```

In this example:

- `cat logfile.txt`: Displays the content of some log file.
- `grep "error"`: Filters lines containing the word “error.”
- `wc -l`: Counts the number of lines.

By chaining these commands with pipes, you efficiently search for errors and obtain the count in a single command, showcasing the power of Unix pipes in orchestrating command-line tasks

9.7.2 Example 2

Count the number of unique numbers in column 2 in the file `data.txt` (above):

```
cut -f 2 -d ',' data.txt | sort | uniq | wc -l
```

- `cut -f 2 -d ',' data.txt`: Uses `cut` to extract the second field (`-f 2`) from each line in `data.txt`. The delimiter is set as a comma (`-d ','`). This step isolates specific data from each line.
- `sort`: Sorts the extracted data alphabetically.
- `uniq`: Removes duplicate lines from the sorted data. This is useful when you want to find unique values in a column.
- `wc -l`: Counts the number of lines in the final output. The result represents the count of unique values in the second column of the original data.

9.8 Exercises

1. List all files in the current directory, including hidden ones. (Hint: first identify which command should be used, then use `man` to look up arguments for including hidden files.)
2. Create a new directory named `unix_exercise` and move into the folder.
3. Redirect the output of the command `echo "Hello, World!"` to a file named `output.txt`. Verify it worked, then remove the file it with `rm`.
4. Add the file `data.txt` (given above) to your folder. Count the number of lines in the `data.txt` the file and print the result.
5. Write the 5 first lines into a file `data_first5.txt`.
6. Write the 5 last lines into a file `data_last5.txt`.
7. Sort `data.txt` numerically in based on the numbers in column 1, and output it to `sorted_data1.txt` (column 2 doesn't have to be sorted).
8. Sort `data.txt` numerically in based on the numbers in column 2, and output it to `sorted_data2.txt` (Hint: use pipelines and streams).
9. Move out of the folder `unix_exercise` and remove the folder and its contents.

Chapter 10

Data Analysis with Pandas, Numpy, and Scipy

IT IS NOT ALLOWED TO USE MODULES THAT DOES NOT COME WITH PYTHON'S STANDARD DISTRIBUTION IN YOUR PROJECTS. THUS THE MODULES DESCRIBED HERE ARE FORBIDDEN IN YOUR PROJECTS. FOR MOTIVATION, SEE THE LAST SENTENCE OF THE CHAPTER

Python has become one of the most widely used languages in data science and scientific computing. This is largely due to a collection of libraries that provide efficient data structures and powerful numerical methods. The three foundational libraries are *NumPy*, *Pandas*, and *SciPy*, which together form the scientific computing ecosystem on which most higher-level modules rely or, at least, were inspired by.

10.0.1 Installation

NumPy, SciPy and Pandas are large libraries and do not come with the standard Python distribution. Perhaps you have these libraries already from previous courses or from following the installation instructions in ???. You can test this by opening the terminal in VS code as View -> Terminal, then type `python` in the terminal window followed by `import pandas`, `import numpy`, and `import scipy`.

If you get an error, they have to be installed separately. You can do this through with Python's package installer `pip` by opening the terminal in VS code as View -> Terminal and then type `pip install numpy scipy pandas`. Or install `conda` and type `conda install scipy` (NumPy and Pandas will be installed automatically).

10.0.2 NumPy

NumPy (Numerical Python) provides the basic building blocks for numerical computation. At its core is the `ndarray`, a fast and memory-efficient multidimensional *array* type implemented in the language C. Conceptually, an *array* is like a Python list, but it stores only one data type and arranges the values in a compact, fixed-size block of memory on the computer, so that numerical operations can be performed much faster. This makes NumPy essential for tasks that involve linear algebra, matrix operations, random number generation, or any large-scale data analysis situation where performance matter.

Many scientific libraries, including Pandas, SciPy, and scikit-learn, are built directly on top of NumPy and rely on its internal representation. While NumPy excels at numerical operations, it does not provide convenient tools for working with tabular datasets with named columns, missing values, or mixed data types common in real-world analysis. This is where Pandas does the job.

10.0.3 Pandas

Pandas introduces two primary structures: the `Series`, representing a labeled one-dimensional data array, and the `DataFrame`, a flexible two-dimensional table resembling an Excel sheet. The `Series` data structure uses the `ndarray` type from NumPy to store the data, but adds additional features on top of the array such as allowing non-integer indexing and element access (think dictionaries, although the datastructures differ). Pandas makes it easy to load and clean data from a variety of sources such as CSV and Excel files, filter and transform columns, merge datasets, handle missing values, and compute descriptive statistics. It has become the standard tool for exploratory data analysis and data preprocessing.

Library	Data Structure	Typical Use
NumPy	<code>ndarray</code>	Numerical computation
Pandas	<code>DataFrame</code> , <code>Series</code>	Tabular data analysis

10.0.4 SciPy

Also building on NumPy, SciPy serves as a comprehensive scientific computing toolbox. While NumPy focuses on arrays and basic operations, SciPy implements advanced algorithms used in mathematics, engineering, and the natural sciences. These include modules for optimization, statistics, signal processing, and more. For example, SciPy allows you to fit regression models and run statistical hypothesis tests. Many of these algorithms would be complex and error-prone to implement manually, and SciPy offers reliable, well-tested implementations frequently used in academic and industrial research.

10.1 Data analysis with Pandas, Scipy, and NumPy)

Let us consider the data from `sample4.txt` from lab 3, but with an added header and removed whitespaces. Create a directory `data` and a file `data.txt` in the `data` folder with the below content.

```
batch,x,y,val
3,0.5,0.5,10
1,-0.1,-0.2,9
2,0.6,-0.15,9
1,-0.5,0.1,0
3,0.5,0.6,22
3,0.2,0.6,9
3,-1,1,2
3,0.3,0.5,45
3,0.5,0.8,23
3,-0.8,-0.8,2
1,0.1,0.2,73
1,0.11,0.1,19
1,-0.1,0.2,72
```

```

1,0.11,-0.1,291
3,0.13,0.9,5
3,0.5,0.1,19
1,-0.9,0.2,-44
1,0.53,-0.2,23
1,0.1,-0.9,55
1,-0.6,-0.2,12
2,0.23,0.01,-17
2,-0.8,-0.5,99
2,-0.6,-0.15,82
2,-0.23,-0.3,63
3,1,1,10
2,-0.93,-0.01,77
2,0.8,-0.5,-2
2,0.23,-0.3,1
2,0.93,-0.01,53

```

First, let us read in the data into a DataFrame (df) with Pandas

```

import os
from matplotlib import pyplot
import pandas as pd

data_file = os.path.join('data', 'data.csv')
results_df = pd.read_csv(data_file)
xdata = results_df['x'] # extracting the x values
ydata = results_df['val'] # extracting the measurement values

```

10.1.0.1 Simple vizualization

Let us plot the values with Matplotlib.

```

pyplot.scatter(xdata, ydata)
pyplot.title("Sample4")
pyplot.xlabel("x")
pyplot.ylabel("Val")
pyplot.savefig("plot-sample4.pdf")

```

10.1.0.2 Statistics

To fit a regression line with Scipy, we can import the sub-module stats from Scipy

```

from scipy import stats
slope, intercept, r_value, p_value, std_err = stats.linregress(xdata, ydata)
print("Slope = ", slope)
print("Intercept = ", intercept)
print("R-squared = ", r_value**2)
print("P value = ", p_value)
print("Standard error = ", std_err)

```

Should give

```
Slope = -5.404139743056493
Intercept = 35.392322523857786
R-squared = 0.0027902734168649885
P value = 0.7855159105779598
Standard error = 19.66140552447426
```

We can also plot the regression line on top of the scatter plot (assuming the code are all in the same script)

```
# Plot regression line
pyplot.plot(xdata, line, label="Linear fit")

# Labels and title
pyplot.title("Sample4")
pyplot.xlabel("x")
pyplot.ylabel("Val")
pyplot.legend()
pyplot.savefig("plot-sample4-linear_fit.pdf")
```

10.1.0.3 Group-by Statistics: Compare Batches

So far we have only analyzed the x-coordinade against the value column for all data. We notice that the data come from different batches. Thanks to Pandas DataFrame structure who indexes the data by the header in the csv file, we can group, split, or select different columns in the DataFrame using the method groupby. Firs we reload the data into a DataFrame

```
# relaod the sample file into a new variable df
df = pd.read_csv(os.path.join("data", "data.csv"))
```

Let us plot a scater plot with colors per batch.

```
colors = {1: 'red', 2: 'green', 3: 'blue'}
df.plot.scatter(
    x='x',
    y='val',
    c=df['batch'].map(colors),      # distinct colors
    figsize=(8,5),
)

# Code to generate a legend
import matplotlib.patches as mpatches
batches = df['batch'].unique()
pyplot.legend(handles=[mpatches.Patch(color=colors[b], label=f'Batch {b}') for b in batches])

pyplot.title("Scatter Plot Colored by Batch")
pyplot.savefig("scatter-by-batch.pdf")
```

It doesn't look like there is a noticable trend or difference in the value (y-axis) per batch. We can assess that further with some statistics.

```
# Group by batch and compute multiple statistics
group_stats = df.groupby("batch").agg(
```

```

    mean_val=("val", "mean"),
    std_val=("val", "std"),
    median_val=("val", "median"),
    max_val=("val", "max"),
    min_val=("val", "min"),
    count=("val", "count")
)

print(group_stats)

```

batch	mean_val	std_val	median_val	max_val	min_val	count
1	51.000000	91.445187	21.0	291	-44	10
2	40.555556	43.017761	53.0	99	-17	9
3	14.700000	13.132233	10.0	45	2	10

We can do testing on whether the means differ significantly using the pairwise t-tests from the scipy stats module.

```

from itertools import combinations # this module is part of the standard distribution

for (a, b) in combinations(df['batchID'].unique(), 2):
    vals_a = df[df['batchID'] == a]['val']
    vals_b = df[df['batchID'] == b]['val']
    t, p = stats.ttest_ind(vals_a, vals_b)
    print(f"Batch {a} vs {b}: t={t:.3f}, p={p:.3e}")

```

Gives:

```

Batch 3 vs 1: t=-1.243, p=2.300e-01
Batch 3 vs 2: t=-1.814, p=8.734e-02
Batch 1 vs 2: t=0.312, p=7.586e-01

```

10.1.0.4 Outlier Detection Using NumPy (Z-Scores)

NumPy provides some basic operations on arrays with numbers.

```

import numpy as np

vals = df["val"].values
z_scores = (vals - np.mean(vals)) / np.std(vals)

df["z_score"] = z_scores
outliers = df[abs(df["z_score"]) > 2.5] # threshold of 2.5 SD

print("Outliers found:")
print(outliers)

# visualize
pyplot.scatter(df["x"], df["val"], label="data")
pyplot.scatter(outliers["x"], outliers["val"], label="outliers", s=100)

```

```

pyplot.legend()
pyplot.title("Outlier Detection Using Z-scores")
pyplot.savefig("outlier_detection_z_scores.pdf")

```

10.1.0.5 Fix formatting errors (e.g., letter O instead of 0 on y-axis)

We have only analyzed x-axis values against the feature 'val' in this chapter using some of the common statistics and plotting functionality offered by Pandas, SciPy, and NumPy. In case we want to also analyze the y-axis values we need to deal with malformed values, as you may remember from lab 3. Below shows sample code to, in this particular case, replace the letter O with a zero. The code first converts the columns to strings, then replaces uppercase letter O with 0 (zero), and finally converts the strings back to numbers. If a value cannot be interpreted as a number, pandas coerces it to NaN instead of raising an error.

```

# Fix common typo: letter 'O' used instead of digit '0' (only in numeric fields)
# Convert columns to strings, replace 'O'-'>'0', then coerce to numeric.
for col in ["x", "y", "val"]:
    # ensure we operate on strings first, then replace, then to_numeric
    df[col] = pd.to_numeric(df[col].astype(str).str.replace("O", "0", regex=False),
                           errors="coerce")

```

10.2 Why you still need classical coding skills

We have only scratched the surface of the data-analysis functionality offered by Pandas, SciPy, and NumPy. However, they are useful only when data are tabular, and there are many types of data and computational problems that cannot be easily interperded as such.

10.2.1 Non-tabular and large data

For example, in bioinformatics we have Fasta and Fastq files to represent sequences. These data analysis modules fails here because:

1. FASTA is not row-based, it is hierarchical, with header information followed by the sequence (possibly many lines).
2. There is no fixed schema; sequences can be of different lengths.

Also, while several file formats in bioinformatics are tabular based, e.g., SAM, GTF, VCF, they are often very large and cannot be loaded in full. Thus, they require customized file processing.

10.2.2 Algorithms

Most computational problems are not solved by tabular data processing. Below are two of the most basic bioinformatic problems taught in a algorithmic bioinformatics course.

10.2.2.1 Example 1: Computing GC Content in Sliding Windows

Given a DNA sequence, compute GC content in windows of size k across the genome:

```

def sliding_gc(sequence, k):
    gc_values = []

```

```

for i in range(len(sequence) - k + 1):
    window = sequence[i:i+k]
    gc = (window.count("G") + window.count("C")) / k
    gc_values.append(gc)
return gc_values

```

This is not a row or column operation, it requires algorithmic windowing on a string. For example, Pandas has no built-in sliding-window for strings, so we would still need Python loops or comprehensions.

10.2.2.2 Example 2: Finding Open Reading Frames (ORFs)

Finding Open Reading Frames (ORFs) is the process of identifying stretches of DNA that could potentially encode proteins. This involves scanning a DNA sequence for start and stop codons in all reading frames. ORF detection is a fundamental step in genome annotation and gene prediction.

This computational problem requires pattern detection with nested loops and modulo arithmetic, and as in previous example, no rows or columns exist, only a long string.

```

def find_orfs(seq):
    """
    Find Open Reading Frames (ORFs) in a DNA sequence.

    This function scans the given DNA sequence for all open reading frames
    in the forward strand. An ORF is defined as a sequence starting with
    the start codon 'ATG' and ending with one of the stop codons ('TAA',
    'TAG', 'TGA'), where the stop codon is in the same reading frame as the start.

    Parameters
    -----
    seq : str
        A string representing the DNA sequence (e.g., 'ATGCGT...').

    Returns
    -----
    list of tuple
        A list of tuples (start, end), where 'start' is the index of the
        first nucleotide of the start codon, and 'end' is the index
        immediately after the stop codon (Python slicing convention).

    Notes
    -----
    - Only the forward strand is considered.
    - Overlapping or nested ORFs may not be fully captured because the
      start codon list is cleared after each stop codon.
    - Sequence indexing is zero-based.
    """
    starts = []
    orfs = []
    n = len(seq)

```

```

for i in range(n - 2):
    codon = seq[i:i+3]
    if codon == "ATG":
        starts.append(i)

    if codon in ("TAA", "TAG", "TGA"): # stop codons
        for s in starts:
            if (i - s) % 3 == 0:
                orfs.append((s, i+3))
            starts = [] # reset for next region
return orfs

```

10.2.2.3 In general

Pandas is good for working with tables and computing statistics (mean, standard deviation, etc.), but it is not designed for general algorithmic problems. Most scientific computing involves custom logic and is algorithm-heavy, not just CSV manipulation.

For example, bioinformatics requires understanding algorithms, sequences, and patterns. **This is why classical programming skills such as loops, conditionals, functions, and basic data structures (e.g., lists and dictionaries) are essential and unavoidable.**

Appendix A

Sample solutions

A.1 Chapter 1

1. x and y switch values

2. For example $(x - y - y)**2$

3. $s*16$

```
4. s = 'hej'
   s2 = s+s
   s4 = s2+s2
   s8 = s4+s4
   s16= s8+s8
   print(s16)
```

```
5. name = input("What is your first name?")
   surname = input("What is your last name?")
   print('Hi', name, surname + '!')
```

6. The numbers are concatenated because they are stored as strings. To actually add the numbers, you have to convert the strings to integers.

```
age = input("Age?")
nr = input("Favorite number?")
print(int(age)+ int(nr))
```


A.2 Chapter 2

1. b, c

```
2. num = int(input("Enter an integer: "))
   if num % 2:
       print("Number is odd!")
   else:
       print("Number is even!")
```

3.

x	y	x and y	x or y
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

4. An example:

```
x = True
y = False
z = False
```

gives

```
True
False
True
False
```

```
5. def division(x, y):
    if x == 0 and y == 0:
        print("Indeterminate form...")
    elif y == 0:
        print("The result is infinite...")
    else:
        return x/y
```

```
6. False
   True
   True
   True
```

```
7. def nand(x, y):
    return not (x and y)

   def nor(x, y):
       return (not x) and (not y)
```

```
def xnor(x, y):  
    return ((not x and not y) or (x and y))  
  
# alternativ xnor  
def xnor(x, y):  
    return (x==y)
```

8. If the functions are defined as above, then we'll get for example:

```
xnor(47, 33)
```

```
33
```

Because both 47 and 33 are true, you might have expected the result to be True, but the functions in the definition of xnor can return other values when their arguments are not of type bool.

A.3 Chapter 3

1. The list [1, 2, 'a', 'b']
2. It is not possible for example to compare strings and floats or ints.

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

```
3. def vowels(s):  
    vs = "AEIOUYaeiouy"  
    out = ""  
    for v in s:  
        if v in vs:  
            out += v  
    return out
```

```
4. def vowels_or_cons(s, save='v'):  
    vs = "AEIOUYaeiouy"  
    out = ""  
    if save != 'c' and save != 'v':  
        print("illegal argument to save")  
        # Here we should raise error. More about error control later  
        return  
  
    for v in s:  
        if save == 'v' and v in vs:  
            out += v  
        elif save == 'c' and v not in vs:  
            out += v  
  
    return out
```

5. If we leave out the block

```
if term != "":  
    out.append(term)
```

then the last word will not be added if the string does not end with a space. If we leave out `if term != ""` but leave out `out.append(term)` at indentation level 1, then an empty string will be added to the end if the string ends with a space. Note also that our `split` only handles one space; it will give an incorrect result if there are several spaces in a row. We can fix this by testing if `term` is empty as below:

```
def split(s):  
    out = []  
    term = ""  
    for c in s:  
        if c == " ":  
            # Do not add term if empty (End up here when consecutive spaces)  
            if term != '':  
                out.append(term)
```

```

        term = ""
    else:
        term += c

    # add the final word unless it is empty
    #if term != "":
    out.append(term)

    return out

```

6. The problem is that we did not use `.copy()` on `list` when we create `list2`. If we do not do this, `list` and `list2` will simply be different names for the same list, so if we change one then the other will change as well. Test the following instead:

```

list = [1,2,3]
list2 = list.copy()
list2.reverse()
print(list)

```

7. Alternative 1:

```

mylist = ["Ho", "ho", "ho!"]
m = mylist.copy()
while m:
    x = m.pop()
    print(x)

print(mylist)

```

Alternative 2:

```

mylist = ["Ho", "ho", "ho!"]
n = len(mylist)
while n > 0:
    print(mylist[n-1])
    n -= 1

print(mylist)

```

Alternative 3:

```

mylist = ["Ho", "ho", "ho!"]
for i in mylist[::-1]:
    print(i)

print(mylist)

```

8. With **break**:

```

10
9

```

With **continue**, everything but 7 will be printed. With **pass**, all the numbers will be printed.

9. For example:

```
mylist = []
while True:
    ans = int(input("Enter a single digit (quit with 0): "))
    # if 0 is *not* to be included in the list.
    if ans == 0:
        break
    else:
        mylist.append(ans)
print(mylist)
```

Another option is to use the `:=` operator, which is a relatively new addition to Python (version ≥ 3.8 , so use this operator with care). The `:=` operator evaluates *and* assigns in an if statement, so we can avoid one line in the code block:

```
mylist = []
while ans := int(input("Enter a single digit (quit with 0): ")):
    mylist.append(ans)
print(mylist)
```

Here `ans` is first assigned the input value and the if condition is evaluated *afterwards*. Because `0` coerces to `False` (`bool(0) == False`), we will exit the while-loop when the user enters `0`.

10. For example:

```
def naiveGCD(a,b):
    for i in range(min(a,b), 0, -1):
        if a % i == 0 and b % i == 0:
            return i

a = 12
b = 3
print(naiveGCD(a,b))
```

or:

```
def naiveGCD(a,b):
    i = min(a,b)
    while i > 0:
        if a % i == 0 and b % i == 0:
            return i
    i -= 1
```

11. For example:

```
def portkod(pw = '1337'):
    curr = ""
    while True:
        n = input("Enter digit:")
        # make sure our string is only 4 digits long
        if len(curr) >= 4:
            curr = curr[1:]
```

```
        print(curr)
        curr += n # add new digit
    if curr == pw:
        print("door unlocked")
        break

portkod(pw = '1337')
```

```
def portkod(pw = '1337'):
    curr = ""
    while True:
        n = input("Enter digit:")
        # make sure our string is only 4 digits long
        if len(curr) >= 4:
            curr = curr[1:]
            print(curr)
        curr += n # add new digit
        if curr == pw:
            print("door unlocked")
            break

portkod(pw = '1337')
```

A.4 Chapter 4

1. For example:

```
a = ['a', 'b', 'c']
b = [1, 2, 3]

# alt 1
d = {}
for i in range(len(a)):
    d[a[i]] = b[i]

# alt 2
d = {a[i] : b[i] for i in range(len(a)) }
```

2.

```
def naiveGCD(a,b):
    for i in range(min(a,b), 0, -1):
        if a % i == 0 and b % i == 0:
            return i

print("x, y, GCD")
for x in range(2, 21):
    for y in range(2, 21):
        gcd = naiveGCD(x, y)
        print(x, y, gcd)
```

3. Replace `for y in range(2, 21):` with `for y in range(x, 21):`

4.

```
def vowels(s):
    vs = "AEIOUYaeiouy"
    d = {vs[i] : i for i in range(len(vs))}
    out = ""

    for v in s:
        if v in d:
            out += str(d[v])
        else:
            out += v
    return out

print(vowels("Hej"))
```

5.

```
def annotate_word(s):
    out = {}
    term = ""
    cnt = 1
    for c in s:
        if c == " ":
            out[term] = cnt
```

```

        term = ""
        cnt +=1
    else:
        term += c

    # add the final word unless it is empty
    if term != "":
        out[term] = cnt
    return out

```

6. Alternative 1: Write `out[cnt] = term` instead of `out[term] = cnt` Alternative 2: Add the following line at the end of the function.

```

out_reversed = { v : k for k, v in out.items()}

```

7. We can have lists as values:

```

def annotate_word(s):
    out = {}
    term = ""
    cnt = 1
    for c in s:
        if c == " ":
            if term in out:
                out[term].append(cnt)
            else:
                out[term] = [cnt]
            term = ""
            cnt +=1
        else:
            term += c

    # add the final word unless it is empty
    if term != "":
        if term in out:
            out[term].append(cnt)
        else:
            out[term] = [cnt]
    return out

```


A.5 Chapter 5

1. We get a `TypeError`. We can catch it like so:

```
def divide_by_elems(filename,x):
    try:
        x = int(x)
    except TypeError:
        print("divide_by_elems:",
              "Second argument cannot be interpreted as an integer.")
    return

# ... rest of function here
```

2. For example:

```
def divide_by_elems(filename,x):
    try:
        x = int(x)
    except TypeError:
        print("divide_by_elems:",
              "Second argument cannot be interpreted as an integer.")
    return
divs = []

    try:
        h = open(filename, 'r')
    except IOError:
        print("divide_by_elems: A file-related problem occurred.")

    for n in h:
        try:
            frac = x / int(n)
            divs.append(frac)
        except ZeroDivisionError:
            print("divide_by_elems: Division by zero.")
        except ValueError:
            print("At least one line contains characters that",
                  "cannot be converted with int().")
    h.close() # close the file
    return divs
data = divide_by_elems('numbers.txt', 2)
print(data)
```

- 3.

A.6 Chapter 6

1. For example:

```
import random
random_integers = [random.randint(1, 100) for i in range(10)]
random_floats = [random.uniform(1, 100) for i in range(10)]
```

2. For example:

```
def is_perfect(x):
    sum = 0
    for i in range(1, x):
        if(x % i == 0):
            sum = sum + i
    if sum == x:
        return True
    else:
        return False

def is_prime(x):
    if x >= 2:
        for y in range(2,x):
            if not ( x % y ):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is prime:", is_prime(x))
    print("The number is prime:", is_perfect(x))

x = int(input("Provide a number to analyze: \n"))
main(x)
```

3. We can add something like

```
choice = input(
    "Type 'G' to manually enter number or 'R'"
    "to randomly generate number: \n")
if choice == "G":
    x = int(input("Provide a number to analyze: \n"))
elif choice == "R":
    x = random.randint(1, 100)
else:
    print("You failed")
```

The important of the random module can be at the top of the file, and our new code should sit below the functions `is_perfect(x)` and `is_prime(x)`. We can have it in `main` or separately; it depends on how we want to be able to use `main`. Should it include the new functionality or not? Normally, functions one might want to import should not include interactive functionality (communication

with the user).

4. We divide up the code so that every function takes parameters and returns. We have removed all globally defined variables (seq, k, substrings) so that we can reuse each individual function in other programs.

```
# Given a sequence as input find distances
# between the copies of the most frequent substring

def get_substrings(seq, k):
    substrings = {}
    for i in range(len(seq) - k + 1):
        substring = seq[i: i+k]
        if substring in substrings:
            substrings[substring] += 1
        else:
            substrings[substring] = 1
    return substrings

def find_most_freq_substring(substrings):
    max_count = 0
    most_freq_substring = ""
    for substring, count in substrings.items():
        if count > max_count:
            most_freq_substring = substring
            max_count = count
    return most_freq_substring

def find_positions(query_substring, seq):
    positions = []
    k = len(query_substring)
    for i in range(len(seq) - k + 1):
        substring = seq[i: i+k]
        if substring == query_substring:
            positions.append(i)
    return positions

def get_distances(positions):
    distances = []
    for p1, p2 in zip(positions[:-1], positions[1:]):
        distances.append(p2 - p1)
    return distances

## Driver code
def main():
    seq = input("Enter a dna string: ")
    k = 3
    substrings = get_substrings(seq, k)
    max_substring = find_most_freq_substring(substrings)
```

```
positions = find_positions(max_substring, seq)
distances = get_distances(positions)
print(distances)
```

5. For example:

```
import itertools
numbers = [13, 24, 42, 66, 78]
string_numbers = [str(i) for i in numbers]
# permutations
perm = itertools.permutations(string_numbers, r=2)
results = [int(''.join(i)) for i in perm]
```

6. •

A.7 Chapter 7

1. Below code is an implementation solving exercises 1-3. You can also implement the try-except statement (exercise 3) within the for loop in the read_fasta function to be able to print out the specific line where things go wrong. However, this will lead to slower runtime, which is significant for large fasta files.

```
import argparse

def read_fasta(f):
    seqs = {}
    for i, line in enumerate(f):
        if line[0] == ">":
            acc = line.strip()
            seqs[acc] = ''
        else:
            seqs[acc] += line.strip()

    return seqs

def get_kmer_windows(seq, w):
    return [seq[i:i+w] for i in range(len(seq) - w + 1)]

def estimate_gc(kmers):
    return [(kmer.count('C') + kmer.count('G')) / len(kmer) for kmer in kmers]

def get_gene_regions(gc_fractions, y):
    regions = []
    curr_region_start = 0
    for i in range(len(gc_fractions) - 1):
        # gene region starts
        if gc_fractions[i] < y and gc_fractions[i+1] >= y:
            curr_region_start = i+1

        # gene region is over
        elif gc_fractions[i] >= y and gc_fractions[i+1] < y:
            regions.append( (curr_region_start, i+1) )

    return regions

def print_results(chr_id, regions):
    for start, stop in regions:
        print(chr_id, start, stop)

def rc(seq):
    rc_dict = {'A': 'T', 'C': 'G', 'G': 'C', 'T': 'A'}
    s = ""
```

```

for nucl in seq:
    nucl_rc = rc_dict[nucl]
    s += nucl_rc

return s[::-1]

def main(args):

    try:
        assembly = read_fasta(args.assm_file)
    except:
        print("There was an error related to parsing\
              of fasta file. Please check the formatting of the file")

    for chr_id, seq in assembly.items():
        if args.verbose:
            print("Processing chromosome {0}".format(chr_id) )
            kmers = get_kmer_windows(seq, args.w) # returns list of k-mers in order
            gc_fractions = estimate_gc(kmers) # estimates GC content per kmer, returns list of floats
            regions = get_gene_regions(gc_fractions, args.gc_fraction)
            print_results(chr_id, regions)

        if args.use_rc:
            kmers = get_kmer_windows(rc(seq), args.w) # returns list of k-mers in order
            gc_fractions = estimate_gc(kmers) # estimates GC content per kmer, returns list of floats
            regions = get_gene_regions(gc_fractions, args.gc_fraction)
            print_results(chr_id, regions)

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Find gene region \
                                             in a genome assembly through GC content.", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('--w', type=int, default = 50,
                        help='Window size to calculate GC content from')
    parser.add_argument('--gc_fraction', type=float, default = 0.7,
                        help='GC content cutoff value.')
    parser.add_argument('assm_file', type=argparse.FileType('r'),
                        help='input assembly file')
    parser.add_argument('outfile', type=argparse.FileType('w'),
                        help='Output CSV-file')

    # Below args not implemented in this example (see exercise 2)

    parser.add_argument('--use_rc', action="store_true",
                        help='Use of reverse complements to find genes')
    parser.add_argument('--verbose', action="store_true",

```

```

help='Pring diagnostic output to terminal (stdout)')

args = parser.parse_args()
main(args)  # Call the main function if run from terminal

```

2. For example

```

import argparse

#the two functions is_prime and is_perfect here
...
...

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Description", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('x', type=int, help='A number')
    args = parser.parse_args()
    print("The number {0} is prime:".format(n), is_prime(n))
    print("The number is perfect:", is_perfect(n))

```

3. For example by adding the argument nargs="+" as below

```

import argparse

#the two functions is_prime and is_perfect here
...
...

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Description", \
                                     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument('x', type=int, nargs="+", help='')
    args = parser.parse_args()
    for n in args.x:
        print("The number {0} is prime:".format(n), is_prime(n))
        print("The number is perfect:", is_perfect(n))

```

4. For example, if you have the program in exercise 5 in a file called “exercise5.py”

```

import exercise5
x = 6
assert exercise5.is_perfect(x) == True, "6 Should be perfect"
x = -1
assert exercise5.is_perfect(x) == False, "-1 is not perfect"

# decide how your program should behave for wrong data type
x = '91'
try:
    is_perfect(x)

```

```
    assert False, "is_perfect did not throw TypeError for string"
except TypeError:
    pass # worked fine, we want a type error

# some more tests on is_prime
# ...
```


A.8 Chapter 8

1. For example `ls -a` (more details: `ls -lha`)
2. For example `mkdir unix_exercise; cd unix_exercise`
3. `echo "Hello, World!" > output.txt, rm output.txt`
4. Assuming you are already in the folder `unix_exercise`: `mv /your/path/to/data.txt ..` Then `wc -l data.txt`
5. `head -n 5 data.txt > data_first5.txt`
6. `tail -n 5 data.txt > data_last5.txt`
7. `sort -n data.txt > sorted_data1.txt`
8. `sort -k 2 -t ',' -n data.txt > sorted_data2.txt`
9. `cd ..; rm -r unix_exercise`